

TEAMATRIX INDUSTRIAL CONSOLE

Automated 13-Ingredient Precision Tea-Blending, Conveying & Mixing System

Complete Technical & Engineering Documentation — Revision 1.0

Raspberry Pi 5 • 4 × Cytron Motion 2350 Pro (RP2350, CircuitPython) • Python 3 / Tkinter HMI

Generated: 2026-05-22 • Project: TeaDispense_ARPICO

This document is the engineering reference for the TeaMatrix Industrial Console. It is written for firmware engineers, embedded developers, electronics engineers, software developers, and the maintenance team that will operate the line after handover. Every parameter, pin, state, and tunable in the following sections is extracted directly from the actual repository (teamatrix_pro.py, teamatrix_backend.py, code.py, hx711_gpio.py, station_config.json) and is not a generic template.

Document Map

§	Section	Audience
1	Project Overview	All
2	Complete System Architecture	Architects, Leads
3	Complete File & Module Analysis	Software, Firmware
4	Dispensing System Engineering	Control / Firmware
5	Hardware Engineering Details	Electronics
6	HX711 & Load-Cell Deep Analysis	Electronics / Instrumentation
7	GPIO & Pin Architecture	Electronics / Firmware
8	Servo System Engineering	Electromechanical
9	UI/UX & Frontend Architecture	Software / Operators
10	Backend & Software Design	Software
11	Configuration System	Integrators / Techs
12	Engineering Decisions & Tradeoffs	Architects
13	Safety & Reliability Engineering	All
14	Performance & Precision Analysis	QA / Process
15	Future Improvements	Roadmap
16	Appendix (Glossary, Calibration, Maintenance, Tests)	All

1. Project Overview

1.1 What the Machine Is

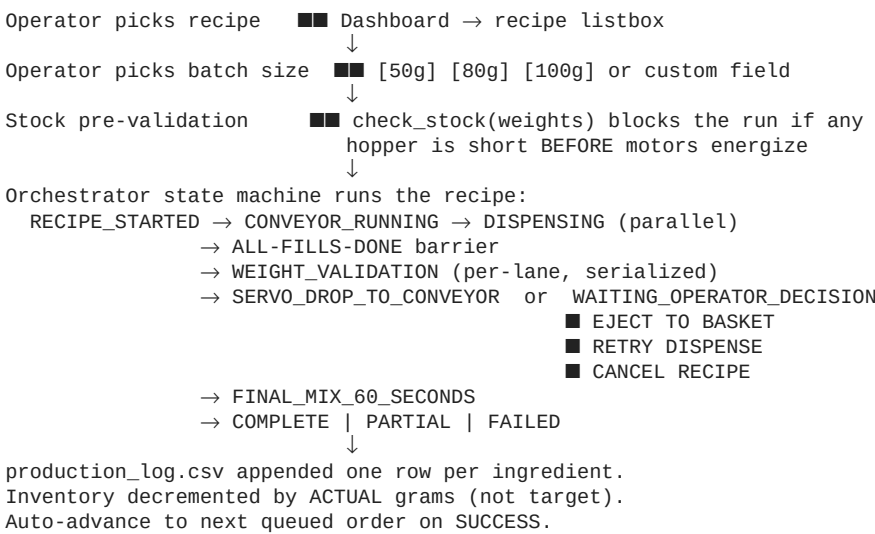
TeaMatrix is a fully-automated, recipe-driven, multi-ingredient tea-blending machine. A Raspberry Pi 5 master controller runs a Tkinter-based touchscreen Human-Machine Interface (HMI) and coordinates four Cytron Motion 2350 Pro RP2350 microcontroller boards over USB-CDC serial. Together they drive 13 independent ingredient stations, a 12 V conveyor belt, and a 12 V industrial mixer to produce a finished tea blend from a single operator action.

Five base teas (Strathspey BOPF, Laxapana Peko, Moray BOP, Silver Tips, Golden Tips) and eight additives/botanicals (Cinnamon Chips, Ginger Pieces, Orange Peel, Lemon Peel, Lemongrass, Rose Petals, Jasmine Petals, Bergamot) are held in 13 hoppers, each with an independent auger driven by a 12 V DC gear motor, a flap controlled by an MG996R servo, and a 100 g load cell connected through a bit-banged HX711 24-bit ADC. The operator selects a recipe, the orchestrator spawns one worker thread per ingredient and dispenses every lane in parallel onto its load cell, validates the weight of each fill against a per-station tolerance band, drops in-tolerance lanes onto a moving conveyor, ejects out-of-tolerance lanes into an inside basket, and runs the mixer for 60 s after the last valid drop. Every order is appended to `production_log.csv`.

1.2 Primary Engineering Goals

- **Precision:** per-station closed-loop dispense with absolute final error bounded to roughly 0.1 g (the loop targets `[target, target + 0.05 g]` and the orchestrator validates within the per-station `scale_tolerance_grams`).
- **Parallelism:** total batch time = `max(individual fill times)`, not sum. 13 concurrent worker threads dispense their stations independently and a barrier synchronises the validation/drop phase.
- **Flexibility:** a single firmware image and single backend handle 13 ingredients with wildly different flow characteristics. Per-station tuning lives in `station_config.json` and is editable from a PIN-gated Behavior tab on the HMI without restarting the application.
- **Robustness:** serial watchdog, HX711 spike rejection, servo cooldown, operator-decision modal, stock pre-validation, and per-lane error containment ensure that one bad ingredient never crashes the line.
- **Reproducibility:** every fill — successful, ejected, retried, cancelled — is recorded with per-ingredient detail in `production_log.csv`; refills are logged to `refill_log.csv`; the order ID auto-increments and persists across reboots.

1.3 Operational Concept (single batch)



1.4 System Capabilities

Capability	Value / Detail
Ingredient stations	13 independent, fully parallel

Capability	Value / Detail
Auger motors	13 × 12 V DC gear motors via Cytron H-bridge (PWM 20 kHz)
Servo flaps	13 × MG996R, 5 V dedicated rail, 50 Hz, 500–2500 µs
Load cells	13 × 100 g, HX711 bit-banged at gain 128
Conveyor	12 V DC, slot 2 on Board 4, duty = 52000 (~80% of 65535)
Mixer	12 V DC, slot 3 on Board 4, duty = 40000 (~61% of 65535)
Serial protocol	115200 baud, newline-terminated ASCII, polled at ~50 Hz
Scale polling	Pi side ~50 Hz; firmware HX711 sample at 25 Hz typical
Watchdog	Firmware: 0.7 s no-command → all motors off
Heartbeat	Pi: W:ALL every 500 ms keeps firmware watchdog reset
Tolerance (default)	±3.0 g per lane, per-station overridable, tolerance(target)=max(0.2, 1% of target) for CSV
Order log	10-column CSV (OrderID, Timestamp, Recipe, Station, Tea, Target_g, Actual_g, Delta_g, Tolerance_g, Status)
Persistence files	station_config.json, recipes.json, inventory.json, ingredients_catalogue.json, production_log.csv, refill_log.csv
Tech-mode PIN	2350 — gates Behavior, Recipe Change, Diagnostic tabs

1.5 Hardware / Software Integration at a Glance

The Pi acts as a stateful orchestrator and persistence layer; the four Motion Pro boards are dumb actuators that only know how to PWM motors, drive servo PWM, and report bit-banged HX711 weights. All state lives on the Pi (station config, recipes, inventory, production log). All real-time motor decisions also live on the Pi — the boards just execute `D:<ch>:<duty>` writes within their 700 ms watchdog window. This split is deliberate: it lets the dispensing intelligence (ML voltage model, spike rejection, micro-tail, top-up, rescue pulses, learned compensation) evolve without re-flashing firmware on four boards.

- Station._fill_t loop uses smoothed weight for stop decisions
- Station.actual recorded at fill end
- Orchestrator._run logs ACTUAL to production_log.csv
- deduct_stock(sid, actual) updates inventory.json
- GUI _loop() shows ui_w on the station card (40 ms refresh)

2.5 State Flow — Aggregate Orchestrator



2.6 State Flow — Per-Lane Worker



CANCEL → `_abort.set()` → FAULT

2.7 Module Relationship Diagram

```
teamatrix_pro.py (UI)
  ■ imports & calls
  ▼
teamatrix_backend.py
  ■■■ INGREDIENTS ←■■ station_config.json      (load/save)
  ■■■ RECIPES     ←■■ recipes.json              (load/save)
  ■■■ inventory   ←■■ inventory.json            (load/save)
  ■■■ INGREDIENT_CATALOGUE ←■■ ingredients_catalogue.json
  ■■■ Board × 4   ■■■ pyserial → /dev/module_0X
  ■■■ Station × 13 ■■■ owns Board ch, applies INGREDIENTS[sid] behavior
  ■■■ Orchestrator ■■■ spawns lane workers, owns state machine
  ■■■ log_production(...) → production_log.csv
  ■■■ refill_station(...) → refill_log.csv

code.py (firmware, identical on all 4 boards, BOARD_ID = 1..4)
  ■■■ pwmio.PWMOut × 4 motors (GP8, GP10, GP13, GP15)
  ■■■ pwmio.PWMOut × 4 servos (GP0, GP1, GP2, GP3)
  ■■■ HX711 × 4 channels via hx711_gpio.py (bit-bang)
  ■■■ ASCII command parser + 0.7 s watchdog
```

2.8 Dispatch Sequence Diagram

[illegible]

3. Complete File & Module Analysis

3.1 Repository Map

File	Lines	Role
teamatrix_pro.py	2,554	Tkinter HMI — 4 operator tabs + 3 tech tabs
teamatrix_backend.py	1,984	Board, Station, Orchestrator, persistence, ML voltage
code.py	305	Motion Pro firmware (identical on all 4 boards)
hx711_gpio.py	182	Bit-bang HX711 driver for CircuitPython
teamatrix_hardware.py	236	Legacy hardware skeleton — earlier ML demo path
teamatrix_config.py	65	Legacy CSV helpers (kept for backward compat)
station_config.json	599	Per-station Behavior values (auto-saved)
inventory.json	53	Per-station container stock (auto-saved)
recipes.json	—	Persisted recipe library (auto-created)
production_log.csv	—	Per-order, per-ingredient log (auto-appended)
refill_log.csv	—	Manual refill events (auto-appended)
99-tea-lanes.rules	18	udev rules → /dev/module_01..04 symlinks

3.2 teamatrix_pro.py — Tkinter HMI (Entry Point)

The single class `App(tk.Tk)` is the application entry point. On construction it pins itself into the backend (`_bk.app_ref = self`) so the backend can schedule UI callbacks via `app_ref.after(0, ...)`. It builds seven notebook tabs, kicks off `init_boards` in a background thread, and starts the 40 ms GUI loop (`_loop`) that drives all visible state.

Key responsibilities

- Render the dashboard: recipe listbox, batch-weight scaler, order queue (max 5), 13 station cards (LN01..LN13) + conveyor card + mixer card, system-log strip, status bar, board status LEDs.
- Render the Behavior tab: per-station tuning grid exposing every field in `INGREDIENTS[sid]` (`mode`, `micro_threshold_g`, `ml toggle`, `decel_factor`, `scale_tolerance_grams`, `fall_delay_seconds`, `target_offset_g`, `servo envelope`, `servo sequence`, `precision-dispensing knobs`, `final_micro_amount`, `settling_delay_s`).
- Render the Recipe Change tab: add / rename / delete / save recipes; ingredient-row editor that maps Station ID → grams.
- Render the Diagnostic tab: per-station manual MOTOR ON, DISP TEST (5 g default), DROP TEST, EJECT TEST, with a recipe-running guard so manual tests cannot collide with an in-flight order.
- PIN-gate Behavior / Recipes / Diagnostic via `_toggle_tech` (`TECH_PIN = '2350'`). `_on_tab_changed` snaps back to Dashboard if the operator lands on a locked tab without authentication.
- Show the out-of-tolerance decision modal `show_drop_decision(payload)` with EJECT/RETRY/CANCEL buttons and keyboard shortcuts (E, R, C, Esc).
- Display toast notifications top-right for queue events and orchestrator outcomes (SUCCESS, PARTIAL, FAILED).
- Auto-refresh the production log when an order completes (`on_done()` → `_refresh_log()`).

Notable design choices

- All UI state updates are scheduled via `app_ref.after(0, ...)` from worker threads. Tkinter is single-threaded; never touch widgets from non-main threads.
- `STATUS_COLORS` is a string → hex map keyed on `Station.status` / `Orchestrator.state`. Adding a new state requires only a new entry there.
- `_dashboard_refresh()` is non-destructive: it does not stop motors or abort recipes; if active, it asks the operator first. This is intentional — refresh must never lose work.

3.3 teamatrix_backend.py — Application Layer

All hardware logic, dispensing intelligence, persistence, and the recipe orchestrator live here. The UI imports symbols from this module and the module owns a singleton `orch = Orchestrator()` and `stations = {sid: Station(sid) for sid in INGREDIENTS}`.

Globals & constants (selected)

Symbol	Value	Purpose
<code>BOARD_PORTS</code>	<code>1:/dev/module_01 ... 4:/dev/module_04</code>	Stable udev symlinks per board
<code>BAUD_RATE</code>	115200	Pi↔board serial speed
<code>POLL_INTERVAL_S</code>	0.02	Per-board poll cadence (50 Hz)
<code>WATCHDOG_INTERVAL_S</code>	0.5 s	Pi-side heartbeat to firmware
<code>HARD_WD_S</code>	0.7 s	No-response soft-stop threshold
<code>SUPPLY_VOLTAGE</code>	12.0 V	Motor PSU; used to convert $v \rightarrow \text{duty}$
<code>ML_A, ML_B, ML_C</code>	-0.0677, 1.6538, 0.5615	Quadratic ML voltage model
<code>VOLT_MIN / VOLT_MAX</code>	2.0 / 6.0 V	Motor cruise voltage clamp
<code>ACCEL_STEP</code>	0.2 V/tick	Acceleration ramp per 20 ms tick
<code>PULSE_VOLTAGE</code>	2.8 V	Default micro-pulse voltage
<code>EMA_ALPHA</code>	0.7	Weight EMA filter speed
<code>MED_BUF_LEN</code>	5	Median-filter window length
<code>MAX_JUMP_G</code>	25 g	Spike-rejection threshold
<code>SPIKE_TOLERATE_N</code>	3	Consecutive jumps before accept
<code>MIXER_DURATION_S</code>	60 s	Mixer runtime after last valid drop
<code>CONVEYOR_DUTY</code>	52000 (≈80%)	Conveyor PWM duty
<code>MIXER_DUTY</code>	40000 (≈61%)	Mixer PWM duty
<code>DROP_TOLERANCE_G</code>	3.0 g	Legacy global ± tolerance
<code>TECH_PIN</code>	'2350'	Technician unlock PIN
<code>OPERATOR_DECISION_TIMEOUT_S</code>	600 s	Auto-CANCEL window for modal
<code>LOW_STOCK_THRESH</code>	100 g	Yellow warning threshold
<code>MAINT_WARN_HOURS</code>	500 h	Motor runtime maintenance alert

Major classes

- `Board(bid, port)`: owns one pyserial connection, runs a `poll()` thread that sends `W:ALL`, parses `WA:<w1>:<w2>:<w3>:<w4>`, and updates `Board.weights[ch]`. Tracks `latency_ms`, `_err` counter per channel, `_soft_stopped` flag. ERR-recovery auto-tares a channel after `ERR_RETRY` (≈3) consecutive ERR reads. `_soft_stop` broadcasts X to every board if no valid response is seen for `HARD_WD_S` = 0.7 s.
- `Station(sid)`: one per ingredient. Wraps `Board` reads via the `raw / smooth()` property pair, owns the full per-station behavior (mode, thresholds, pulse durations, servo sequence, safety envelope, learned compensation), implements `_fill_t` (the precision two-phase fill loop), `_drop_t` (smooth sweep + tap + reset + detach), `_eject_blocking` (back-flip into reject basket), `test_motor_pulse / test_dispense_blocking` for the Diagnostic tab, and rate-limited / range-clamped / cooldown-aware `_servo()` and `_sweep()` helpers.

- Orchestrator: singleton, owns the recipe state machine. `dispatch()` validates stock and spawns `_run()` in a daemon thread. `_run()` drives `RECIPE_STARTED` → `CONVEYOR_RUNNING` → `DISPENSING` → `MIXING` → `FINAL_MIX_60_SECONDS` → `COMPLETE` / `PARTIAL` / `FAILED`. `_lane_worker()` is the per-ingredient worker; `_ask_operator()/submit_decision()` implement the out-of-tolerance modal with a 600 s timeout. `abort()` is the safe shutdown: sets `_abort` event, releases the all-fills-done barrier, kills all motors, and reverts state to `IDLE`.

Hardware abstraction

Two free functions, `set_motor(bid, ch, duty)` and `set_servo(bid, ch, angle)`, are the ONLY direct hardware commands in the module. They translate to `D:<ch>:<duty>\n` and `S:<ch>:<angle>\n` on the matching Board via `bwrite()`. `estop_all()` broadcasts `X\n` to every board, which the firmware reads as 'all motor duty = 0 immediately'. `set_motor` voltage→duty conversion is `duty = int(v/12.0 * 65535)`.

3.4 code.py — Motion Pro Firmware

Single-file CircuitPython program executed on each Cytron Motion 2350 Pro. Identical across all 4 boards; `BOARD_ID` (1..4) is the only edit before deployment, but the live application doesn't actually rely on `BOARD_ID` because device-path ordering is fixed by the udev symlinks (`99-tea-lanes.rules`).

Responsibilities

- Initialise 4 × `pwmio.PWMOut` for motors (frequency 20 kHz, duty 0) and 4 × `pwmio.PWMOut` for servos (frequency 50 Hz). Servo angle 0..180° is mapped linearly to duty 1638..8192 (≈500 μs..2500 μs pulse).
- Initialise 4 × `HX711` channels via `hx711_gpio.HX711(clk, dout, gain=128)`, `set_scale(REF_UNIT[ch])`, tare 5 samples. Board 4 only initialises channel 1 (Bergamot); channels 2/3/4 are used for the Conveyor / Mixer / spare DC motors and have no load cell.
- On boot, home all servos to 0° then detach (PWM signal off) so they do not sit holding load — also the same behavior used at the end of every drop/eject sequence on the Pi side.
- Main loop: process newline-terminated ASCII commands from `sys.stdin`. Commands: `W:ALL` (returns `WA:<w1>:<w2>:<w3>:<w4>`), `T:ALL`, `T:<n>`, `D:<n>:<duty>`, `S:<n>:<angle>` (angle=-1 detaches), `X` (e-stop), `ID?` (returns 'Board <BOARD_ID>'). Unknown commands are silently ignored.
- 0.7 s watchdog: if no valid command arrives within `WATCHDOG_TIMEOUT_S`, all motor PWMs are set to duty 0. The Pi heartbeat (`W:ALL` every 500 ms) keeps this timer reset under normal operation.

Hardware abstraction

Function	Hardware effect
<code>stop_all_motors()</code>	Set <code>motors[0..3].duty_cycle = 0</code> (E-STOP, watchdog)
<code>set_motor_duty(ch, duty)</code>	Clamp duty to 0..65535 and write <code>motors[ch-1].duty_cycle</code>
<code>angle_to_duty(angle)</code>	Linear 0..180° → 1638..8192 (≈500..2500 μs at 50 Hz)
<code>set_servo_angle(ch, a)</code>	<code>a == -1</code> → <code>duty=0</code> (detach); else duty from <code>angle_to_duty</code>
<code>read_all_weights()</code>	Return list of 4 strings: '<gx>', 'ERR' (HX711 timeout), 'OFF' (no scale)

3.5 hx711_gpio.py — Bit-bang HX711 Driver

Bare-metal CircuitPython driver for the HX711 24-bit ADC. The class exposes `set_gain(128|64|32)`, `read(timeout_s)`, `read_average(times, outlier-drop when n≥5)`, `tare(times)`, `set_scale(REF_UNIT)`, `get_units(times)`, `power_down()`, `power_up()`.

Timing details

Parameter	Value	Note
<code>_CLK_HALF_PERIOD</code>	1 μs	Datasheet minimum is 0.1 μs; 10× margin chosen for RP2350 + CircuitPython overhead

Parameter	Value	Note
Default gain	128	Channel A, 1 extra clock pulse after the 24-bit read
Read timeout	0.5 s	If DOUT does not go LOW within 0.5 s, returns None (treated as ERR upstream)
Outlier rejection	min+max dropped when len $\geq$ 5	Reduces influence of spikes inside the firmware-side average
Pull-up on DOUT	Pull.UP	HX711 DOUT is open-drain; pull-up needed for clean reads

Why bit-bang and not hardware SPI?

- HX711 is not a standard SPI peripheral — its protocol is 'wait for DOUT LOW, then clock 24 bits MSB-first plus 1–3 gain-select pulses'. There is no chip-select line, no MOSI; only CLK and DOUT. Bit-banging is simpler and avoids fighting SPI controllers that expect CS/SS or MOSI.
- The RP2350 has more than enough headroom at 1 μ s half-period — even with CircuitPython overhead, a 24-bit read completes in well under 1 ms.
- Each channel uses its OWN clock and data pin (no clock sharing). This matters because the HX711 latches gain selection at the end of each conversion via extra clock pulses; sharing a clock across multiple HX711s would force them all to the same gain and would interleave their DOUT lines on a shared bus. Independent clocks = independent gain settings + independent conversion timing per channel.

3.6 teamatrix_hardware.py — Legacy Hardware Skeleton

An earlier, simpler hardware coordinator that predates the current Orchestrator / Station design. It exposes a `HardwareCoordinator` class that discovers boards by sending 'ID?\n' and reading the response, polls each board at 25 Hz, broadcasts 'X\n' for E-STOP, and runs a single `dispatch_recipe()` loop with the original 'parallel + tolerance 0.2 g check + drop + 30 s mix' flow.

- `calculate_ml_voltage(flow_rate)` — the same quadratic model used in `teamatrix_backend.solve_v()`, expressed in inverse form for direct voltage output rather than the quadratic-solve for `v` given target flow.
- `force_eject(station_num)` / `force_tare(station_num)` — manual UI helpers.
- Watchdog implementation here is a 500 ms 'HBT' broadcast rather than the W:ALL heartbeat used by the production code. The HBT command is not recognised by the current firmware; it would simply be ignored.
- Kept in the tree as a reference for the simpler 'first-cut' flow and for backwards-compatible regression testing.

3.7 teamatrix_config.py — Legacy CSV Helpers

Companion to `teamatrix_hardware.py`. Holds the original 13-station mapping in a `STATIONS` dict, the `CONVEYOR_CONFIG` / `MIXER_CONFIG` slots, and a `get_next_order_id()` + `log_order(order_id, data)` pair that writes a 5-column CSV (Timestamp, OrderID, IngredientName, TargetWeight, ActualWeight). The production code path now uses `log_production()` in `teamatrix_backend.py`, which writes a richer 10-column CSV.

3.8 station_config.json — Per-Station Behavior

Plain JSON, one top-level entry per station ID ("1" through "13"). Loaded at import time by `load_station_config()`, merged with the `_DEFAULTS` map, validated and clamped by `_apply_behavior_defaults()`, and exposed as the global `INGREDIENTS` dict. Saved by `save_station_config()` each time the Behavior tab is committed. The fill loop reads live values via `Station.apply_behavior()` so changes take effect on the very next dispense without an application restart.

Every field is range-clamped on load (see `load_station_config` in `teamatrix_backend.py`): tolerances 0.05..50 g, fall delay 0..30 s, target offset $\pm$ 5 g, servo angles $-90..+120^\circ$, pulse durations 5..500 ms, max top-up pulses 1..50, etc. Out-of-range or non-numeric values fall back to the documented defaults so an operator can never put the machine into an unsafe state by editing the JSON by hand.

3.9 inventory.json — Container Stock

13 entries keyed by station ID, each {capacity: float, stock: float}. Default capacity 1000 g per container. Auto-saved on every dispense (deduct_stock by ACTUAL grams) and every refill (refill_station, appends a row to refill_log.csv). The Inventory tab refreshes every ~2 s. Below LOW_STOCK_THRESH (100 g) the station label flips to '■ LOW' and the station card border turns amber.

3.10 production_log.csv — Order Log

10-column CSV. Header: OrderID, Timestamp, Recipe, Station, Tea, Target_g, Actual_g, Delta_g, Tolerance_g, Status. One row per ingredient per order. Order ID auto-increments and persists across reboots — _load_last_oid() reads the last row and bumps from there. Even cancelled and ejected orders are written with status_override set so the log always reflects what the machine actually did.

3.11 99-tea-lanes.rules — udev Symlinks

Stable device paths /dev/module_01..04 are created from the four boards' USB serial numbers (idVendor 239a, idProduct 8111, the Cytron Motion 2350 Pro identifiers). Without these symlinks, Linux would assign /dev/ttyACMx based on plug-in order, which can swap across reboots and silently break the Pi↔Board mapping.

4. Dispensing System Engineering

4.1 Dispense-Mode Selection Tree

Every station has a `dispense_mode` field set from the Behavior tab. The fill loop derives `use_pulse` from mode and target:

```
use_pulse = (mode == 'micro')
            OR (mode == 'auto' AND target <= micro_threshold_g)

mode='normal'      → use_pulse = False  (legacy bulk + topup)
mode='micro'       → use_pulse = True   (always pulse)
mode='auto', tgt ≤ 5 g → use_pulse = True   (PURE MICRO)
mode='auto', tgt > 5 g → use_pulse = False (NORMAL + SETTLE + FINAL_MICRO)
```

4.2 Mode Comparison

Mode	Phases	When chosen	Pros	Cons
normal	COARSE → SLOWDOWN → SETTLE → FINAL_MICRO → TOP-UP → RESCUE	Forced legacy mode; dense free-flowing ingredients (BOPF, Peko)	Fastest bulk speed; ML model fully engaged; learned compensation tuned per fill	Risk of overshoot if learned_compensation_g drifts; relies on top-up + rescue to clean up
micro (pulse-only)	Pulse → motor off → settle → stable_read → repeat until eff_tgt	Forced for every fill regardless of size; sticky / chunky ingredients (cinnamon bark, bergamot, ginger pieces)	Lowest overshoot risk; predictable per-pulse gain; fine ingredients land softly	Slow at large targets (≥ 30 g would take many pulses); no bulk advantage
auto (default)	Pure micro for $\text{tgt} \leq \text{thr}$; normal+settle+final_micro for $\text{tgt} > \text{thr}$	Default for all 13 stations	Best of both — bulk speed when target is big, pulse precision when target is tiny	Threshold tuning matters: too low and a ~6 g fill goes bulk and overshoots; too high and a ~20 g fill is slow

4.3 Phase 1 — Normal Coarse (target > micro_threshold_g)

When the fill cannot be done purely with pulses, the auger runs in continuous mode against a cruise voltage derived from the ML model.

```
flow_guess = 6.0 g/s if target < 15 g else 9.0 g/s
cruise     = solve_v(flow_guess)                if ml_model_enabled else VOLT_MAX
cv         = VOLT_MIN (start)

while station_weight < bulk_stop:                bulk_stop = tgt - final_micro_amount_g
  if w >= slow_start (= bulk_stop - 1g):
    frac      = (w - slow_start) / MICRO_TAIL_SLOWDOWN_G
    cv_target = cruise * (1 - 0.6 * frac)          # decay to 40 % of cruise
    cv        = clamp_toward cv_target by accel_step (status=SLOWDOWN)
  else:
    cv        = min(cruise, cv + accel_step)      # accelerate (status=FILLING)
    _motor(cv)
    sleep 20 ms                                   # 50 Hz inner loop
  _stop()    # motor off, flow_rate_obs recorded for diagnostics
```

4.4 Phase 2 — Settle + Stable Read

After bulk stop, the loop pauses for `settling_delay_s` (default 1.0 s) with the motor off, then performs a `stable_read`: `smooth()` is sampled at ~25 Hz until `stable_sample_count` (default 4) consecutive samples fall within `stable_window_g` (default 0.05 g) of each other. The function returns the latest stable reading — not the pre-pulse reading — so any fall-delay material from the auger is included.

4.5 Phase 3 — FINAL_MICRO (pulse-stop-wait-check loop)

Universal late-fill stage used by both pure-micro and normal modes. Each pulse is a single short motor-on burst at `PULSE_VOLTAGE` (2.8 V default), pulse duration adapted to the remaining gap:

Gap to eff_tgt	Pulse duration	Constant
> 2.0 g	110 ms (default)	pulse_ms_large
> 0.8 g	60 ms	pulse_ms_medium
> 0.3 g	35 ms	pulse_ms_small
≤ 0.3 g	20 ms	pulse_ms_tiny
≤ 0.5 g (extra-careful regime)	pulse_ms_tiny + voltage × MICRO_FINE_VOLT_FACTOR (0.8)	MICRO_FINE_GAP_G

After each pulse the loop waits `settling_delay_s` for the material to actually land, then `stable_read` measures the new weight. Predictive stop: if $(\text{current} + \text{avg_gain}) \geq \text{stop_at}$ the loop exits without firing another pulse. Stall guard: 3 consecutive pulses with < 5 mg gain → loop exits with a 'stalled' warning (auger empty or hopper bridged).

4.6 Phase 4 — TOP-UP (after FINAL_MICRO)

Same pulse-stop-wait-check structure as `FINAL_MICRO` but with a different stop semantic: it runs at `PULSE_VOLTAGE` only and is bounded by `max_topup_pulses` (default 30 per station). It is the standard guard against residual undershoot in normal mode.

4.7 Phase 5 — RESCUE (chronic undershoot)

If TOP-UP exits and the lane is still ≥ 0.10 g (`TOPUP_RESCUE_TRIGGER_G`) short of `eff_tgt`, the loop fires up to 8 (`TOPUP_RESCUE_PULSES`) extra pulses at `TOPUP_RESCUE_VOLTAGE` = 3.4 V. Rescue pulses are never shorter than 40 ms — they are deliberately stronger than the regular pulse voltage to overcome ingredient-specific stickiness that defeated the main top-up loop.

4.8 Learned Compensation (adaptive overshoot bias)

At the end of every NORMAL-mode fill, the loop updates the per-station `learned_compensation_g` via an EMA on the residual error $d = \text{actual} - \text{target}$:

```
new_learn = learned_c + LEARN_ALPHA * d          # LEARN_ALPHA = 0.30
new_learn = clamp(new_learn, ±PRECISION_LEARNED_CLAMP_G) # ±3.0 g cap
→ persisted to station_config.json via _persist_learned_comp(sid, new_learn)
```

A persistent overshoot of, say, +0.3 g causes the next fill to cut earlier by approximately 0.09 g (0.3×0.30). After a handful of fills the bias converges and the residual error drifts toward zero. This is why the very first fill of a fresh container can be slightly off while subsequent fills tighten up.

4.9 Target Offset ("stop early")

Positive `target_offset_g` reduces the effective target so the fill stops short of the recipe target. Example: target = 5 g, offset = 0.5 g → `eff_tgt` = 4.5 g. The same `eff_tgt` is propagated through `COARSE`, `SLOWDOWN`, `SETTLE`, `FINAL_MICRO`, `TOP-UP`, and `RESCUE` so the whole loop respects the 'land at `target - offset`' goal. Used to compensate for known post-cutoff in-flight gravity. Clamped to ± 5 g for safety.

4.10 Spike-Reject / Median / EMA Pipeline

`Station.smooth()` runs every `smooth()` call (i.e. every fill-loop tick and every UI tick). It is the only place raw weight enters the decision loop, so its filtering directly bounds dispense accuracy.

```
raw                ← Board.weights[ch]   (latest WA: response)
↓
debounce           ← if Δt < SCALE_DEBOUNCE_S (40 ms): return last EMA
↓
spike rejection    ← if buffer primed and |raw - last_kept| > MAX_JUMP_G:
                    spike_streak++; drop sample
                    if spike_streak >= SPIKE_TOLERATE_N: ACCEPT
                    (real step change, e.g. operator placed a weight)
↓
median over 5 samples ← deque(maxlen=MED_BUF_LEN)
↓
EMA at α = 0.7      ← ema = ema*(1-α) + med*α (first sample replaces ema)
↓
dead band          ← return 0.0 if |ema| < ZERO_RANGE (0.50 g)
```

↓
returned value

4.11 Conveyor / Mixer Interlocks

- Conveyor must be running before any servo drop. The drop() method on Station refuses to fire if conveyor_running is False (logs 'DROP BLOCKED — conveyor interlock not satisfied'). The orchestrator turns the conveyor on at RECIPE_STARTED and only off after FINAL_MIX_60 completes, so under normal operation the interlock is always satisfied.
- Mixer starts on the FIRST valid drop, not at recipe start. This minimises wear on the mixer motor when an entire recipe gets cancelled or all lanes are ejected (mixer simply never starts). _note_valid_drop sets mixer_started=True under _mixer_lock and timestamps last_drop_t.
- FINAL_MIX_60_SECONDS counts 60 s from the LAST valid drop, not from the first. A multi-ingredient recipe whose last lane finishes 12 s after the first will still see a full 60 s mix cycle.

4.12 Ingredient-Family Behavior Examples

Family	Examples	Typical mode	Why
Dense, free-flowing	Strathspey BOPF, Laxapana Peko, Moray BOP, Silver/Golden Tips	auto (bulk + final_micro)	Predictable flow rate, ML model converges, large targets (85–94 g) get bulk speed; FINAL_MICRO 3 g handles the tail
Chunky / irregular	Cinnamon Chips, Ginger Pieces, Bergamot	auto with conservative auto_micro_tail / final_micro	Per-pulse gain varies by orientation of the chunks; pulse mode tolerates that better than continuous
Low-density botanicals	Rose Petals, Jasmine Petals, Lemongrass	auto, but recipe targets are small (3–5 g) → PURE MICRO path	Petals stick to chute walls and to each other; bulk would over- or under-dose. Pulses + settle delay let each batch land before the next pulse
Peels	Orange Peel, Lemon Peel	auto, mid-range targets	Behave like a hybrid: 4–5 g tail uses pulse, bulk uses normal

Rose petals and jasmine petals are the canonical low-density / sticky case. With target = 4 g (Silver Tips Rose & Citrus recipe), the loop enters PURE MICRO from 0 g, uses 110/60/35/20 ms pulses depending on the gap, and applies the extra-careful MICRO_FINE_GAP_G regime (tiny 20 ms pulse at $0.8 \times \text{PULSE_VOLTAGE} = 2.24 \text{ V}$) for the last 0.5 g. Settling_delay_s of 1.0 s is critical: petals can take 600–900 ms to fully settle after a pulse, and reading the scale too soon over-estimates the live weight and under-doses the final delivery.

4.13 Comparative Engineering Notes

Question	Answer & reasoning
Micro vs Normal dispensing	Micro trades speed for accuracy — every pulse is independently verified by a settle + stable_read, so overshoot is bounded by the per-pulse gain (typically 50–150 mg). Normal trades accuracy for speed — continuous flow at cruise voltage covers large targets in seconds but relies on ML-predicted cutoff and a top-up tail to land within tolerance.
Pulse vs Continuous	Pulses give the system a chance to measure between energizations, which is mandatory for low-density / sticky ingredients where the settling time exceeds the response time of a continuous controller.

Question	Answer & reasoning
ML-assisted vs Fixed-threshold	Fixed thresholds (constant cruise voltage) work for dense free-flowing tea but waste time at small targets. The quadratic ML model ($-0.0677 f^2 + 1.6538 f + 0.5615$) maps a desired flow rate to a voltage; the fill loop chooses 6 g/s for $tgt < 15$ g and 9 g/s otherwise. This single parameter ('how fast do I want to flow?') absorbs ingredient-specific gain into the recipe-level guess.
Low-density vs Heavy ingredients	Heavy ingredients (BOPF) have high per-pulse gain (often 200–400 mg/pulse at $2.8 V \times 40 ms$) — pulses are usable but slow at 85 g. Light ingredients (petals) have very small per-pulse gain — pulses are necessary because continuous flow drops 1–3 g in a single tick.

5. Hardware Engineering Details

5.1 Component Inventory

Component	Qty	Spec / Part
Raspberry Pi 5	1	8 GB RAM, USB-C PD 5 V/5 A dedicated supply
Cytron Motion 2350 Pro	4	RP2350-based, CircuitPython 10+, 4× DC motor H-bridge, 4× servo PWM, USB-CDC
DC gear motor (auger)	13	12 V, driven via Motion Pro H-bridge, PWM 20 kHz
Servo motor	13	MG996R, 5 V, 50 Hz, pulse 500–2500 μs
Load cell	13	100 g rated, full Wheatstone bridge
HX711 ADC	13	24-bit, gain 128, bit-banged via Motion Pro GPIO
DC motor (conveyor)	1	12 V, on Board 4 motor slot 2 (D:2:<duty>)
DC motor (mixer)	1	12 V, on Board 4 motor slot 3 (D:3:<duty>)
12 V PSU	1	≥20 A capacity, common +/- with everything except Pi
5 V buck converter	1	≥10 A, dedicated to servo rail only

5.2 Raspberry Pi 5 — Master Controller

- Role: Tkinter HMI, four pyserial threads (one per board), 13 station objects, orchestrator state machine, persistence (JSON / CSV).
- Why a Pi 5 and not a 4: improved USB host scheduling for the four simultaneous CDC links, better Tkinter performance at 1440×900, more headroom for the per-station EMA / median / spike-reject pipeline.
- Power: dedicated USB-C PD 5 V / 5 A. The Pi must never share its supply with the 12 V motor rail or the 5 V servo rail — motor back-EMF and servo inrush will crash the Pi.
- Storage: SD card. `log_production()` does an `os.fsync(f.fileno())` after every CSV append so the log survives a hard power-cut.

5.3 Cytron Motion 2350 Pro — Per-Board Controller

- Microcontroller: RP2350 (dual Cortex-M33 @ 150 MHz) running CircuitPython 10+.
- Onboard hardware: 4× DC motor H-bridge channels, 4× servo headers, 3V3 logic rail (used to power HX711 modules), USB-CDC for both REPL and user-app serial.
- Identical firmware (`code.py` + `hx711_gpio.py`) on every board — board identity is purely a function of which USB port enumerates first. `udev` rules (`99-tea-lanes.rules`) bind specific serial numbers to `/dev/module_01..04` so the assignment is stable across reboots.
- Why not a Pi-side GPIO breakout instead: four independent boards isolate fault domains (a stuck HX711 on Board 2 cannot stall Board 1's poll loop) and physically de-route 13 channels of motor noise away from the Pi.

5.4 DC Gear Motors (augers)

- 12 V DC gear motors, one per ingredient station. The gearing is sized to deliver auger torque adequate for dense BOPF/Peko at the upper end of the cruise voltage band (`VOLT_MAX` = 6 V at runtime).
- Driven via the Motion Pro H-bridge at 20 kHz PWM (`MOTOR_PWM_FREQ` in `code.py`). 20 kHz is above human hearing and well above the motor mechanical resonance window — silent operation and minimal

acoustic noise on the load cells.

- Duty cycle resolution: 16-bit (0..65535). Voltage→duty: $\text{duty} = \text{int}(v / 12.0 \times 65535)$. VOLT_MIN/VOLT_MAX = 2.0/6.0 V → roughly duty 10 922 to 32 768 in normal operation; conveyor 52000 (~80 %), mixer 40000 (~61 %).

5.5 Servo Motors

The README documents two compatible models; both fit the project's PWM mapping (1638..8192 duty at 50 Hz = ≈500..2500 μs):

Field	MG996R	MG90S
Stall torque	~10 kg·cm @ 6 V	~1.8 kg·cm @ 5 V
Stall current	≥ 1.4 A at 6 V (continuous risk)	~0.6 A at 5 V
Recommended PSU per servo	≥ 2 A peak	≥ 1 A peak
Use case here	Flap + tap sequence under loaded hopper	Smaller flaps / prototypes

MG996R / MG90S servos are not designed for continuous holding torque. The firmware homes them to 0° at boot then detaches (duty_cycle=0) so they do not draw stall current at rest. Every drop/eject sequence ends with a SERVO_DETACH_SETTLE_S (0.15 s) wait followed by a -1 sentinel (detach). See section 8 for the full servo safety story.

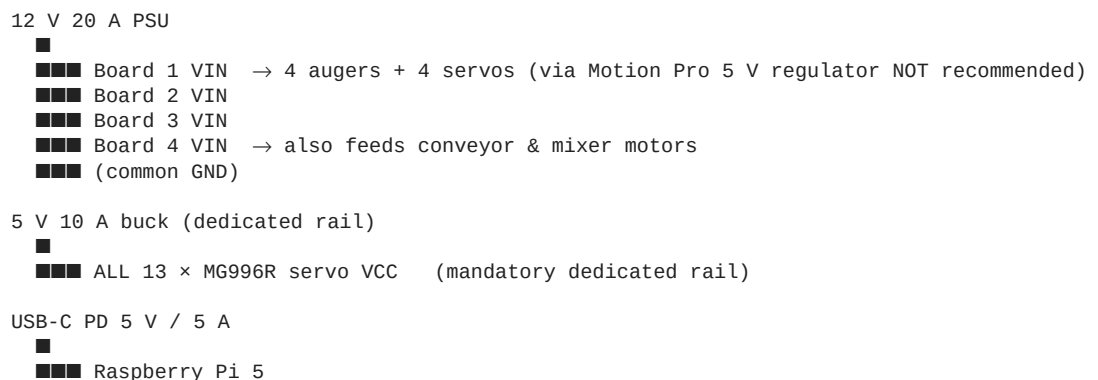
5.6 Conveyor & Mixer

- Both are 12 V DC motors with no scale, no servo. Wired into motor slots 2 and 3 on Board 4. Their duty constants live in teamatrix_backend.py (CONVEYOR_DUTY = 52000, MIXER_DUTY = 40000).
- Conveyor starts at recipe start (RECIPE_STARTED) and runs through the entire dispense + mix window. It is the cooperating mechanism for the servo drop: tilting the flap before the conveyor is moving would deposit material into a static cup; tilting after the conveyor is at speed places material onto a moving belt that immediately transports it under the mixer head.
- Mixer starts on the first valid drop (not at recipe start) to spare wear when an entire recipe is ejected or cancelled. Runs for 60 s after the LAST valid drop.

5.7 Load Cells

- 100 g full-scale, full-bridge strain gauge. One per station, mounted below the ingredient cup that catches material from the auger.
- Excitation comes from the HX711 module (4.3 V typical). Differential output ±10 mV range maps to ±2²³ counts on the HX711 at gain 128.
- Per-cell REF_UNIT is fitted at calibration time: a 100 g calibration weight is placed on the empty platform, the raw HX711 reading is captured (via a temporary debug print in code.py), and REF_UNIT = raw / 100.0 is written into the REF_UNITS list in code.py. Each board has its own four REF_UNITS values; mismatched cells must be calibrated individually.

5.8 Power Architecture



Single common GND: 12 V PSU –, 5 V buck –, Pi GND (star topology recommended)

Critical rules (verbatim from the engineering notes in the README):

- Never power servos from the Motion Pro 3.3 V or 5 V rail — servo stall current is well above the regulator's capacity and the spike will brown out the RP2350.
- Never share the 12 V motor rail with the Pi. Motor back-EMF will crash the Pi.
- Star-grounded common return — all three rails (12 V, 5 V servo, Pi 5 V USB) must share a single chassis ground to keep the load-cell differential clean.

6. HX711 & Load Cell — Deep Engineering Analysis

6.1 What the HX711 Does

The HX711 is a 24-bit Σ - Δ analogue-to-digital converter purpose-built for full-bridge resistive sensors (load cells, pressure cells). It combines a chopper-stabilised low-noise instrumentation amplifier (gain 128 / 64 / 32), an integrated regulator for the bridge excitation, an on-chip oscillator, and a simple two-wire serial interface (CLK + DOUT). There is NO standard SPI/I²C — the protocol is non-standard and must be bit-banged or implemented with a state machine.

6.2 Read Protocol (per hx711_gpio.py)

1. Wait for DOUT to go LOW ← signals 'conversion complete'
if not LOW within 0.5 s → return None
2. Clock in 24 data bits, MSB first ← each rising edge of CLK latches one bit on DOUT, sampled by the master
3. Send `gain\_pulses` extra clocks ← 1 = gain 128 channel A (default)
3 = gain 64 channel A
2 = gain 32 channel B
(programs gain for the NEXT conversion)
4. Apply 2's-complement sign-extend ← if raw & 0x800000: raw -= 0x1000000
5. Returned raw int → grams = (raw - OFFSET) / SCALE

Clock half-period in the driver is 1 μ s. The HX711 datasheet allows down to 0.1 μ s; the 10 $\times$ margin avoids CircuitPython scheduler jitter on the RP2350.

6.3 Why separate CLK lines per HX711 (no shared clock)

- Gain is latched per HX711 by the count of extra clocks at the end of the previous conversion. A shared CLK would force all HX711s to the same gain. The codebase uses gain 128 everywhere, but keeping independent CLK lines preserves the ability to choose gain per channel later (e.g. switch a station to gain 64 if it overflows at 128).
- DOUT is open-drain. Multiple HX711s sharing a CLK would still need a separate DOUT per chip; the architecture chosen (one CLK + one DOUT per channel) is the cleanest 1:1 mapping.
- Conversion timing is per-chip — each HX711 runs its own internal oscillator (~80 kHz nominal) and lowers DOUT when its sample is ready. Independent clocks let the firmware poll each channel exactly when its data is ready instead of synchronising 4 chips against a master clock.

6.4 Pin Map per Channel (code.py SCALE_PINS)

Channel	CLK pin (PD_SCK)	DAT pin (DOUT)
1	board.GP4	board.GP5
2	board.GP6	board.GP7
3	board.GP16	board.GP17
4	board.GP26	board.GP27

Each board uses the exact same CLK/DAT mapping; board identity is purely the USB device path. Wiring on the Motion Pro: HX711 VCC → Motion Pro 3V3 rail; HX711 GND → common GND; CLK and DAT to the GPIO pins above.

6.5 Sampling & Throughput

Property	Value	Note
HX711 internal sample rate	10 SPS / 80 SPS	Rate selected by hardware pin RATE — usually 10 SPS for tea-blending precision
Driver read latency	≤ 25 ms typical at 10 SPS	Limited by HX711 ready time, not CLK speed
Pi-side poll cadence	20 ms (POLL_INTERVAL_S)	Per-board thread issues W:ALL ~50 Hz

Property	Value	Note
Smoothed update rate	~25 Hz at Station.smooth()	SCALE_DEBOUNCE_S = 40 ms caps the upper rate
UI refresh rate	25 Hz (GUI_MS = 40 ms)	Frames the operator sees

6.6 Noise Sources & Mitigation

Noise source	Effect	Mitigation in this codebase
Motor PWM EMI	Sub-gram ripple on DOUT, occasional MAX_JUMP	20 kHz PWM (ultrasonic), star ground, MAX_JUMP_G = 25 g spike reject, vibe_frozen() freezes scale polling on boards 3/4 while mixer is on
Servo current draw on shared 5 V rail	Brownout on HX711 VCC during servo motion	Servos powered from dedicated 5 V buck rail (NOT Motion Pro regulator); HX711 powered from Motion Pro 3.3 V (decoupled)
Load-cell thermal drift	Slow zero baseline shift over minutes	Operator-initiated 'TARE ALL' button at session start; per-station tare on demand
Cable pickup / RF	Random single-sample outliers	Median over MED_BUF_LEN = 5 samples plus EMA at $\alpha = 0.7$
Hopper vibration on conveyor / mixer start	False weight transient	set_vibe_freeze(True) suspends Board 3/4 polling for VIBE_FREEZE_S = 2 s on mixer start

6.7 Calibration Procedure (per channel)

- Ensure the cell is unloaded and the ingredient cup is empty.
- Send T:<n>\n from the UI or a serial terminal — firmware tare averages 5 samples.
- Place a 100 g calibration weight on the cup.
- Read the raw HX711 value (temporarily print hx.read() from code.py).
- $REF_UNIT = raw_reading_with_100g / 100.0$.
- Update the REF_UNITS list in code.py for that channel and redeploy.
- Repeat for all 13 channels (4 per board, 1 on Board 4).

6.8 Sensitivity, Drift, and Precision Limits

Property	Value	Note
Cell rating	100 g full scale	FS chosen for tea ingredient targets up to ~95 g
ADC resolution	24-bit (signed)	16 777 215 LSB across $\sim \pm 20$ mV at gain 128
Theoretical LSB	$\sim 12 \mu g$	Limited by amplifier noise; effective resolution ~ 5 –20 mg
Effective per-sample noise	≈ 5 –20 mg ($1-\sigma$)	Median + EMA reduce this further at the cost of latency
Practical accuracy goal	± 0.05 g landed	Achieved by the predictive cut + settle + stable_read pipeline
UPPER_TOL_G	0.05 g	The fill loop targets [target, target + 0.05 g]
Per-station scale_tolerance_grams	3.0 g (default)	Used by the orchestrator's WEIGHT_VALIDATION; smaller tolerances tighten the drop-or-eject decision

6.9 Power-Down / Power-Up

- `power_down()` holds SCK HIGH for $> 60\ \mu\text{s}$ $\rightarrow$ HX711 enters low-power mode (typical $1\ \mu\text{A}$).
- `power_up()` drops SCK LOW for 1 ms then re-issues `set_gain(128)` to re-latch the gain register.
- Not used in the production flow because the boards are always powered; provided in the driver for diagnostic / future low-power use.

7. GPIO & Pin Architecture

7.1 Per-Board GPIO Map (common to all 4 boards)

Function	GPIO	Pin signal type	Voltage	Owner module	Notes
Scale 1 CLK	GP4	Digital out (bit-bang)	3.3 V	hx711_gpio.HX711	Timing-critical (1 μ s half-period)
Scale 1 DAT	GP5	Digital in, pull-up	3.3 V	hx711_gpio.HX711	Open-drain DOUT; ready=LOW
Scale 2 CLK	GP6	Digital out	3.3 V	hx711_gpio.HX711	
Scale 2 DAT	GP7	Digital in, pull-up	3.3 V	hx711_gpio.HX711	
Scale 3 CLK	GP16	Digital out	3.3 V	hx711_gpio.HX711	
Scale 3 DAT	GP17	Digital in, pull-up	3.3 V	hx711_gpio.HX711	
Scale 4 CLK	GP26	Digital out	3.3 V	hx711_gpio.HX711	
Scale 4 DAT	GP27	Digital in, pull-up	3.3 V	hx711_gpio.HX711	
Motor 1 PWM	GP8	PWM out, 20 kHz	Drives 12 V H-bridge	pwmio.PWMOut	Channel 1 / Board-4 Bergamot auger
Motor 2 PWM	GP10	PWM out, 20 kHz	Drives 12 V H-bridge	pwmio.PWMOut	Channel 2 / Board-4 Conveyor
Motor 3 PWM	GP13	PWM out, 20 kHz	Drives 12 V H-bridge	pwmio.PWMOut	Channel 3 / Board-4 Mixer
Motor 4 PWM	GP15	PWM out, 20 kHz	Drives 12 V H-bridge	pwmio.PWMOut	Channel 4 (spare on Board 4)
Servo 1 PWM	GP0	PWM out, 50 Hz	3.3 V logic, servo on dedicated 5 V rail	pwmio.PWMOut	Duty 1638..8192 = $\approx$ 500..2500 μ s
Servo 2 PWM	GP1	PWM out, 50 Hz	5 V rail	pwmio.PWMOut	
Servo 3 PWM	GP2	PWM out, 50 Hz	5 V rail	pwmio.PWMOut	
Servo 4 PWM	GP3	PWM out, 50 Hz	5 V rail	pwmio.PWMOut	

7.2 Per-Station Channel Mapping (Pi-side view)

Station	Board	Channel	Ingredient	Motor GPIO	Servo GPIO	Scale CLK/DAT
1	1	1	Strathspey BOPF	GP8	GP0	GP4 / GP5
2	1	2	Laxapana Peko	GP10	GP1	GP6 / GP7
3	1	3	Moray BOP	GP13	GP2	GP16 / GP17
4	1	4	Silver Tips	GP15	GP3	GP26 / GP27
5	2	1	Golden Tips	GP8	GP0	GP4 / GP5
6	2	2	Cinnamon Chips	GP10	GP1	GP6 / GP7
7	2	3	Ginger Pieces	GP13	GP2	GP16 / GP17
8	2	4	Orange Peel	GP15	GP3	GP26 / GP27
9	3	1	Lemon Peel	GP8	GP0	GP4 / GP5
10	3	2	Lemongrass	GP10	GP1	GP6 / GP7
11	3	3	Rose Petals	GP13	GP2	GP16 / GP17
12	3	4	Jasmine Petals	GP15	GP3	GP26 / GP27
13	4	1	Bergamot	GP8	GP0	GP4 / GP5

Station	Board	Channel	Ingredient	Motor GPIO	Servo GPIO	Scale CLK/DAT
CV	4	2 (motor only)	Conveyor	GP10	—	—
MX	4	3 (motor only)	Mixer	GP13	—	—

7.3 Timing-Critical Pins

- HX711 CLK pins (GP4, GP6, GP16, GP26) need sub-microsecond toggling. At gain 128, a 24-bit read + 1 gain pulse = 25 clock cycles $\times$ 2 μ s $\approx$ 50 μ s of CLK activity; with the wait-for-ready preamble the whole read is $\leq$ 25 ms at the 10 SPS hardware rate.
- Motor PWM pins (GP8, GP10, GP13, GP15) run at 20 kHz. PWM is generated by hardware (pwmio.PWMOut) so the duty-cycle update has no CPU cost and is glitch-free.
- Servo PWM pins (GP0..GP3) run at 50 Hz. At duty 0 the PWM line is held LOW, equivalent to detaching the servo. Servo position is held mechanically by the gearbox while detached; on the next angle command, PWM restarts with the new duty value.

7.4 Physical Voltage Domains

Domain	Voltage	Connected to
RP2350 logic	3.3 V	All GPIO; HX711 VCC; HX711 CLK/DAT
Servo VCC (external)	5 V dedicated rail	All 13 $\times$ MG996R servos
Motor power	12 V	All 13 augers + conveyor + mixer (via H-bridge)
Pi 5 input	5 V USB-C PD	Pi only — never shared

8. Servo System Engineering

8.1 Why Servo Selection Matters

The flap servos are the most failure-prone electromechanical element in the line. They live under a loaded hopper, they reverse direction every drop/tap cycle, and they sit on the same 5 V rail. Choosing the wrong servo, or driving it carelessly, results in burnt windings, stripped plastic gears (MG996R is metal-geared; MG90S is plastic), or brownouts that crash the Pi.

8.2 Drop Sequence (Station._drop_t)

```
servo['drop_angle_start'] → servo['drop_angle_end']      smooth sweep at drop_speed_dt
                        (default 0° → 70°, 10 ms/° = 0.7 s sweep)

hold drop_hold_s (0.5 s)
drop_tap_count taps (default 3):
    tap_low (45°) ← drop_tap_dt 0.18 s → tap_high (70°)
    → repeat 3×
smooth sweep tap_high → return_angle (0°) at return_speed_dt (15 ms/°)
SERVO_DETACH_SETTLE_S (0.15 s) + send angle = -1 (detach)
```

Hardened with:

- `_servo_busy_lock` prevents overlapping drop/eject on the same station
- `servo_move_timeout_s` aborts the sequence after configurable budget
- `try/finally` guarantees servo detach even on exception or timeout

8.3 Eject Sequence (Station._eject_blocking)

```
servo['eject_angle_start'] → servo['eject_angle_end']    smooth sweep at eject_speed_dt
                        (default 0° → -70°, 15 ms/° = 1.05 s sweep)

hold eject_hold_s (0.8 s)
smooth sweep back → return_angle (0°) at return_speed_dt
detach
```

8.4 Hardening: Rate Limit, Direction Pause, Cooldown, Envelope

Mechanism	Constant / setting	Effect
Range clamp	<code>safe_angle_min / safe_angle_max</code> per station (default -90 / +120°)	Any angle outside [min,max] is clamped before being written to the firmware. Absolute limits <code>SAFE_ANGLE_MIN_LIMIT/MAX</code> bound the per-station envelope itself.
Rate limit	<code>SERVO_MIN_CMD_INTERVAL_S</code> = 5 ms	Sleeps if two writes arrive faster than this; prevents flooding the serial bus and the servo driver.
Direction-change pause	<code>SERVO_DIRECTION_CHANGE_PAUSE_S</code> = 50 ms	Inserted whenever motion direction reverses (e.g. 70° → 45° in a tap). Avoids slamming the servo into a direction change.
Cooldown	<code>SERVO_COOLDOWN_RUN_S</code> = 30 s active / <code>SERVO_COOLDOWN_WINDOW_S</code> = 60 s / <code>SERVO_COOLDOWN_DURATION_S</code> = 5 s	If the cumulative active time within a 60 s rolling window exceeds 30 s, the servo is forced to rest for 5 s. Prevents thermal stall on a stuck flap.
Detach after motion	<code>SERVO_DETACH_SETTLE_S</code> = 0.15 s then <code>S:<ch>:-1</code>	Releases the PWM signal so the servo doesn't hold load mechanically at the end of every drop/eject. The MG996R/MG90S gear train holds position passively without drawing current.
Busy lock	<code>Station._servo_busy_lock</code>	Refuses overlapping drop/eject sequences on the same station (rapid-click protection in the UI).
Per-movement timeout	<code>servo_move_timeout_s</code> (per station, default 8 s)	If a sweep takes longer than the budget, the sequence aborts and the servo still detaches in the finally block.

8.5 PWM Mapping (firmware code.py:angle_to_duty)

```
SERVO_FREQ      = 50 Hz
SERVO_MIN_DUTY  = 1638      ≈ 500 µs pulse → angle 0°
SERVO_MAX_DUTY  = 8192      ≈ 2500 µs pulse → angle 180°
duty = int(SERVO_MIN_DUTY + (angle/180) * (SERVO_MAX_DUTY - SERVO_MIN_DUTY))
angle = -1 → duty = 0 (signal off / detach)
```

8.6 Servo Power Considerations

- DEDICATED 5 V rail with at least 10 A budget. MG996R can draw 1.4 A during a stall; thirteen servos moving simultaneously can momentarily demand 5–8 A from the buck.
- 100–470 µF bulk decoupling close to the servo connector strip is recommended to absorb inrush and prevent the rail from sagging when all 13 flaps open during the simultaneous DROPPING phase.
- Star-ground the servo rail to the main chassis GND. Sharing ground with the 12 V motor rail through a long, lossy return path lets motor noise modulate the servo signal and causes intermittent jitter.
- Never power servos from the Motion Pro 5 V or 3.3 V regulator. Those regulators are sized for the RP2350 and the HX711 modules; a single stalling MG996R will brown them out and the board will reboot mid-dispense.

8.7 Why "Servos Burn" (and How This Code Prevents It)

- Continuous holding torque at end-of-travel: the servo PID keeps driving current into the motor against a mechanical stop. Solution: detach (S:<ch>:-1) after every motion. The gearbox holds position passively with zero current.
- Repeated direction reversals: stall current spikes each time the motor reverses against inertia. Solution: SERVO_DIRECTION_CHANGE_PAUSE_S inserted automatically by Station._servo() whenever motion sign flips.
- Cumulative duty cycle: even pulsed motion adds up. Solution: 30 s active in any 60 s window triggers a 5 s forced rest.
- Out-of-range commands: a configuration mistake driving the servo to –120° could mechanically bind the linkage. Solution: per-station safe envelope is clamped first to absolute limits, then to per-station limits, before any write.

9. UI / UX & Frontend Architecture

9.1 Tab Layout

Tab	Zone	Purpose
■ Dashboard	Operator	Recipe picker, batch-weight scaler, order queue (max 5), 13 station cards, conveyor + mixer cards, system-log strip, status bar, board LEDs
■ Inventory	Operator	Per-station stock / capacity, REFILL entry, low-stock badges, auto-refresh ~2 s
■ Prod Log	Operator	production_log.csv viewer (newest first), row-count badge, auto-refresh on order completion
■ Health	Operator	Board connectivity + latency, motor run hours, maintenance alerts at MAINT_WARN_HOURS = 500 h
■ ■ Behavior	Tech (PIN)	Per-station tuning: mode, micro_threshold, accel/decel, ML toggle, tolerance, fall_delay, target_offset_g, servo angles+envelope, precision dispensing knobs (fine_margin, inflight_comp, max_topup, pulse durations L/M/S/T), final_micro_amount_g, settling_delay_s
■ ■ Recipe Change	Tech (PIN)	Recipe CRUD (rename without duplicates, overwrite confirm, ingredient de-dup, list refresh)
■ ■ Diagnostic	Tech (PIN)	Per-station MOTOR ON / DISP TEST / DROP TEST / EJECT TEST, auto-capped duration, recipe-running guard

9.2 Dashboard Anatomy

TEAMATRIX v2.1						E-STOP	[ORDER #1042]	STATE: IDLE	BRD1..4
Recipes						LN01 Strathspey	LN02 Laxapana	LN03 Moray	LN04 ...
(listbox)						85.04g READY	0.00g IDLE	0.00g IDLE	...
						LN05 Golden	LN06 Cinnamon	LN07 Ginger	LN08 ...
Ingredient						LN09 Lemon	LN10 Lemongrass	LN11 Rose	LN12 ...
Targets						LN13 Bergamot			
Batch:						CONVEYOR: IDLE	MIXER: IDLE	30s	QUEUE: 2 / 5
[50][80]						[CONV ON][CONV OFF]	[MIX ON][MIX OFF]		
[100][]									
START						SYSTEM LOG (scrollable)			
PAUSE						14:21:08	Recipe loaded: Ceylon Spiced Breakfast (100g)		
STOP						14:21:11	Order ORD-1042 → Ceylon Spiced Breakfast		
RESUME						14:21:11	Fill → Station 1 Strathspey BOPF 85.0g		
TARE						14:21:32	S1 STABLE w=85.04g err=+0.04g pulses=2		

9.3 Station Card

- Header: 'LN<NN>' station ID + ingredient label.
- Big numeric weight readout (white when within 0.3 g of target, green otherwise). Updates via `_loop()` every 40 ms but only repaints when `|w - last_w| > DEAD_ZONE` (0.01 g).
- Status line: IDLE / TARING / FILLING / SLOWDOWN / SETTLING / MICRO / FINAL_MICRO / PULSING / WAITING / READY TO DROP / ERROR / STOPPED / DISCONNECTED / EJECTING / EJECTED — colour

comes from STATUS_COLORS.

- Progress bar (Tk.DoubleVar): $100 \times (w / \text{target})$ capped at 100 %.
- Inline buttons: START / STOP / DROP / EJ / T — disabled-greyed when not applicable to the current status (managed by Station._btns()).
- Card border turns amber when the station is below LOW_STOCK_THRESH (100 g) and is part of the currently selected recipe; red when the owning Board is disconnected.

9.4 Order Queue

- Max 5 queued orders. Each entry: oid, recipe name, weights dict.
- Queue is rendered as cards in a scrollable canvas (q_inner). Clicking a card opens an edit dialog (recipe + batch weight).
- Order ID is reserved at queue-add time (next_oid()) so the operator sees the actual order ID before the run starts.
- Start picks queue[0] (peek, not pop). Pop happens in on_done() only on SUCCESS or PARTIAL — a FAILED order remains at the queue head for operator retry. Operators can use the explicit DELETE button to drop a failed entry.

9.5 Behavior Tab

PIN-gated, mirrors every field in INGREDIENTS[sid]. _beh_collect validates each entry on Save, _beh_populate fills the form from the selected station, _beh_save_current writes through save_station_config and then refresh_station_behavior() so every Station instance reloads live values without an app restart. A diagnostic 'ACTIVE BEHAVIOR' log line is printed at the start of every fill so the operator can confirm the tab values are actually being honoured.

9.6 Out-of-Tolerance Decision Modal

- Triggered by Orchestrator._ask_operator() when a lane's WEIGHT_VALIDATION fails (delta outside [0, scale_tolerance_grams]).
- Three buttons: ■ EJECT TO BASKET (amber), ■ RETRY DISPENSE (blue), X CANCEL RECIPE (red). Keyboard shortcuts E/R/C/Esc.
- 10-minute timeout (OPERATOR_DECISION_TIMEOUT_S = 600) defaults to CANCEL so an unattended out-of-tolerance lane does not stall the line.
- Modal is serialized across lanes via Orchestrator._modal_lock. Lanes wait for the all-fills-done barrier before any modal can pop, so early-finishing lanes cannot interrupt still-dispensing lanes.

9.7 Theming & Refresh

- Dark palette: BG_DARK #0d0f11, BG_PANEL #111316, BG_CARD #141618, C_GREEN #00e676, C_RED #ff1744, C_AMBER #ffab00 — chosen for readability on a high-DPI touchscreen in industrial light.
- Single _loop() at 40 ms drives all live UI updates (board LEDs, station cards, conveyor/mixer cards, maintenance, inventory refresh every ~2 s, diagnostic single-station panel).
- All cross-thread UI writes go through app_ref.after(0, fn) — Tkinter is single-threaded; the after queue is the only safe path.

10. Backend & Software Design

10.1 Threading Model

Thread	Lifetime	Role
Main / Tk	Process	UI; <code>_loop()</code> every 40 ms; after-queue dispatch
<code>Board.connect() × 4</code>	Once per <code>init_boards()</code>	Open pyserial, send T:ALL, home and detach servos
<code>Board.poll() × 4</code>	Process	20 ms tick: send W:ALL, parse WA:, update <code>Board.weights[ch]</code> , heartbeat, soft-stop watchdog
<code>Station._fill_t</code>	Per fill	Precision multi-phase fill loop for one station
<code>Station._drop_t</code>	Per drop	Servo sweep + tap + reset + detach
<code>Station._eject_blocking</code> thread	Per eject	Servo backflip; called fire-and-forget from UI button
<code>Orchestrator._run</code>	Per recipe	Aggregate state machine; spawns lane workers
<code>Orchestrator._lane_worker × N</code>	Per active lane in recipe	Phase 1 dispense → barrier → phase 2 validate/drop/decide
<code>init_boards</code> subthread	Once at startup	Spawned daemon to avoid blocking the UI

10.2 Locks & Synchronisation Primitives

Primitive	Owner	Purpose
<code>Board.lock</code>	Board	Serialise pyserial reads/writes on one CDC link
<code>Station._servo_busy_lock</code>	Station	Refuse overlapping drop/eject on the same lane
<code>Orchestrator._modal_lock</code>	Orchestrator	Serialise the out-of-tol modal across lanes
<code>Orchestrator._mixer_lock</code>	Orchestrator	Atomic 'first valid drop starts mixer' check
<code>Orchestrator._dispense_done_lock</code>	Orchestrator	Increment the all-fills-done counter
<code>Orchestrator._all_dispense_done</code>	Event	Released when every lane finishes phase 1; gates phase 2
<code>Orchestrator._decision_event</code>	Event	Operator-decision wait (with timeout)
<code>Orchestrator._abort</code>	Event	Hard abort; observed by every loop

10.3 Recipe Execution Engine

`Orchestrator.dispatch(recipe, weights)` is the single entry point. It runs `check_stock` first (atomically blocks any recipe that would exhaust a hopper), then resets every per-run flag, then spawns `_run()` in a daemon thread. `_run()` walks the state machine described in section 2.5 and exits with one of three outcomes — SUCCESS, PARTIAL, FAILED — that the UI's `on_done()` callback uses to drive the queue (auto-advance on SUCCESS, pause for operator on PARTIAL, keep at head on FAILED).

10.4 Queue Management

- List of up to 5 dicts maintained in `App._queue`. Persisted only in-memory; the JSON files do not track the queue (intentional — queue is a session-local concept).
- Auto-advance pattern: `on_done()` pops the order on SUCCESS, calls `self.after(1000, self._auto_next)` which starts the next queued order if the orchestrator is idle. PARTIAL/FAILED suppress auto-advance.

- Stock pre-validation runs in `dispatch()` — a queued order whose stations are short still appears in the queue, but pressing START fails fast and pops a toast 'Order blocked: insufficient stock'.

10.5 Hardware Abstraction Boundary

- Backend never speaks GPIO. Everything bottoms out at `set_motor` / `set_servo` / `bwrite`, which write ASCII to a serial port owned by a Board instance.
- Firmware never speaks state. `code.py` has no concept of recipes, stations, or fills. It is a pure actuator: 'set this duty', 'move to that angle', 'read all four weights'.
- This split is the reason the project can ship multiple dispense-mode strategies and learned compensation without re-flashing four boards every iteration.

10.6 Error Handling Strategy

Failure mode	Detection	Recovery
HX711 timeout on one channel	<code>Board.poll()</code> sees 'ERR' in WA: response	Counter <code>_err[ch]++</code> ; after <code>ERR_RETRY</code> (<code>=3</code>), auto-tare <code>T:<ch></code> , reset counter, log 'auto-tare'
Board stops responding (CDC drop)	<code>Board.poll()</code> sees no valid WA: within <code>HARD_WD_S</code> (0.7 s)	<code>_soft_stop()</code> broadcasts X to every board; log 'SOFT-STOP'. Reconnect via <code>Board.reconnect()</code> (UI Health tab button or restart)
Auger fails to deliver material (hopper bridged)	<code>Station._fill_t</code> micro/top-up/rescue loops detect 3 consecutive pulses with <code>< 5 mg</code> gain	Loop exits with 'stalled' warning; weight remains short of target; orchestrator's <code>WEIGHT_VALIDATION</code> will raise the operator-decision modal
Out-of-tolerance fill	<code>Orchestrator._lane_worker</code> checks <code>0 ≤ delta ≤ scale_tolerance_grams</code>	Operator modal: EJECT to inside basket, RETRY (re-runs fill on same lane), CANCEL (aborts whole recipe)
Operator does not respond to modal	Event timeout in <code>_ask_operator</code> (<code>OPERATOR_DECISION_TIMEOUT_S = 600 s</code>)	Defaulted to CANCEL; recipe enters FAULT state
Insufficient stock at dispatch	<code>check_stock()</code> before spawning workers	<code>dispatch()</code> returns False; UI shows toast and does not change queue
Servo cooldown exceeded	<code>_servo()</code> cumulative active time within 60 s window	Forced 5 s rest; log 'servo cooldown — overheat/stall protection'
Servo move timeout	<code>Station._drop_t</code> / <code>_eject_blocking</code> deadline check	Sequence aborts; status → ERROR; servo detached in finally; lane fails validation downstream
Unexpected exception in recipe loop	try/except around <code>Orchestrator._run</code>	Calls <code>_safe_stop('FAULT')</code> ; aborts orchestrator; sets <code>last_outcome=FAILED</code>

10.7 Logging

- Three log surfaces: (1) the on-screen system-log strip on the Dashboard, fed by `log_msg()`; (2) `production_log.csv` on disk, fed by `log_production()` with one row per ingredient per order; (3) `refill_log.csv` on disk, fed by `refill_station()` with one row per manual refill.
- `log_msg(msg, level)` dispatches to every registered callback in `_log_callbacks` (the UI registers one to repaint the strip).
- `log_production()` does an explicit `os.fsync(f.fileno())` so the CSV survives a hard power-cut. `ensure_log_file()` always recreates the header row, so even an empty / deleted file is immediately viewable.

10.8 Diagnostics

Diagnostic tab routes each test through `Station.test_motor_pulse()` or `Station.test_dispense_blocking()` which both refuse to run if `orch.running` is True or if the owning board is offline. Manual motor pulses are clamped to

MAX_MANUAL_MOTOR_S (3 s) and MANUAL_TEST_VOLTAGE (2.5 V). The single-station Diagnostic panel refreshes live weight and last status every 40 ms via `_diag_single_refresh_static`.

10.9 Configuration Management & Live Reload

- `INGREDIENTS = load_station_config()` loads from `station_config.json` at import time. `_DEFAULTS` provides factory defaults; `_apply_behavior_defaults()` backfills missing keys; the load function clamps every value to its safe range before exposing it.
- `save_station_config(INGREDIENTS)` writes through pretty-printed JSON.
- `refresh_station_behavior()` re-applies every field of `INGREDIENTS[sid]` into the live Station instance for that sid — this is what makes the Behavior tab take effect on the very next dispense.
- `RECIPES = load_recipes() / save_recipes()`; `inventory = load_inventory() / save_inventory()`; `INGREDIENT_CATALOGUE = load_catalogue() / save_catalogue()` — same load/save pattern across all four JSON files.

10.10 Scaling Considerations

- More than 13 stations: requires another Motion Pro board and another udev rule; the orchestrator and station class scale linearly with `INGREDIENTS` size, but the dashboard grid layout currently assumes 13 cards + 2 system cards in a fixed grid.
- Higher polling rate: `POLL_INTERVAL_S` can be reduced toward 10 ms, but the HX711 internal sample rate (10 SPS hardware-set) is the actual bottleneck; the smoother will then waste polls.
- Faster recipes: the current bottleneck is single-recipe sequencing, not lane parallelism. Multiple recipes back-to-back are handled by the queue and auto-advance.
- Remote operation: backend is a synchronous Python module. A FastAPI layer could front it for REST/Web access without touching the fill loop, but careful attention to the GIL / thread / event-loop interaction is required.

11. Configuration System

11.1 Per-Station Behavior Schema

Every entry in `station_config.json` mirrors the fields produced by `save_station_config()` in `teamatrix_backend.py`. The schema below is extracted verbatim from the codebase.

Field	Type	Default	Range / Clamp	Purpose
label	str	Strathspey BOPF	—	Display label on the dashboard card
ingredient_name	str	= label	—	Logical name (for the catalogue dropdown)
dispense_mode	str	auto	{auto, micro, normal}	Mode selection (see §4)
micro_threshold_g	float	5.0 g	—	Targets $\leq$ this go PURE MICRO in auto mode
accel_step	float	0.2 V/tick	—	Voltage ramp per 20 ms in COARSE phase
decel_factor	float	1.5	—	(reserved, kept for back-compat)
ml_model_enabled	bool	True	—	Toggle <code>solve_v()</code> ; off $\rightarrow$ cruise = VOLT_MAX
servo	dict	DEFAULT_SERVO	see §11.2	Drop / eject / tap sequence
scale_tolerance_grams	float	3.0 g	0.05 .. 50.0 g	Orchestrator validation band
fall_delay_seconds	float	2.0 s	0 .. 30 s	Post-cutoff settle for in-flight
target_offset_g	float	0.0 g	± 5.0 g	Stop-early offset (positive = stop short)
safe_angle_min	float	-90°	$-90 .. +120^\circ$	Per-station servo lower envelope
safe_angle_max	float	$+120^\circ$	$-90 .. +120^\circ$	Per-station servo upper envelope
servo_move_timeout_s	float	8.0 s	1 .. 60 s	Per-sequence movement budget
coarse_margin_g	float	5.0 g	0 .. 100 g	Legacy COARSE $\rightarrow$ FINE handoff margin
fine_margin_g	float	0.30 g	0 .. 20 g	FINE stop margin before in-flight
inflight_compensation_g	float	0.15 g	0 .. 20 g	Static post-cutoff in-flight
learned_compensation_g	float	0.0 g	± 3.0 g	Adaptive per-station overshoot bias
settle_ms	int	500	0 .. 5000 ms	Post-stop wait before stable read
stable_window_g	float	0.05 g	0.005 .. 1.0 g	Width of the stable-read window
stable_sample_count	int	4	2 .. 20	Consecutive samples to declare stable
pulse_ms_large	int	110 ms	5 .. 500 ms	Pulse width when gap > 2.0 g
pulse_ms_medium	int	60 ms	5 .. 500 ms	Pulse width when gap > 0.8 g

Field	Type	Default	Range / Clamp	Purpose
pulse_ms_small	int	35 ms	5 .. 500 ms	Pulse width when gap > 0.3 g
pulse_ms_tiny	int	20 ms	5 .. 500 ms	Pulse width when gap ≤ 0.3 g
max_topup_pulses	int	30	1 .. 50	TOP-UP pulse budget
auto_micro_tail_g	float	3.0 g	0 .. 25 g	Legacy AUTO-mode pulse tail length
final_micro_amount_g	float	3.0 g	0 .. 10 g	BULK stops this far before target
settling_delay_s	float	1.0 s	0 .. 30 s	Motor-off wait between BULK and FINAL_MICRO

11.2 Servo Sub-Schema

Field	Default	Range	Meaning
drop_angle_start	0°	−90..+90°	Servo starting angle for the drop sweep
drop_angle_end	70°	0..+120°	End of the drop sweep (forward tilt)
drop_speed_dt	0.010 s/°	0.005..0.05	Time per degree in the drop sweep
drop_hold_s	0.5 s	0.5 s	Hold time at drop_angle_end
drop_tap_count	3	0..10	Number of tap oscillations after drop
drop_tap_low	45°	0..90°	Low extreme of the tap oscillation
drop_tap_high	70°	0..120°	High extreme of the tap oscillation
drop_tap_dt	0.18 s	0.05..0.5	Time between tap end-points
eject_angle_start	0°	−90..+90°	Eject sweep starting angle
eject_angle_end	−70°	−120..0°	Eject sweep end (backward tilt into basket)
eject_speed_dt	0.015 s/°	0.005..0.05	Time per degree in the eject sweep
eject_hold_s	0.8 s	0.5 s	Hold at eject_angle_end
return_angle	0°	−10..+10°	Final neutral angle
return_speed_dt	0.015 s/°	0.005..0.05	Time per degree on the return sweep

Cross-field validation in `_coerce_servo()` ensures `drop_tap_high > drop_tap_low` (otherwise `tap_high` is bumped) and `eject_angle_end < eject_angle_start` (otherwise `eject_end` is forced to start − 30°). Missing or non-numeric fields fall back to `DEFAULT_SERVO`.

11.3 Recipe File (recipes.json)

```
{
  "Ceylon Spiced Breakfast": { "1": 85.0, "6": 10.0, "7": 5.0 },
  "Citrus Earl Grey Style": { "2": 92.0, "8": 5.0, "9": 3.0 },
  ...
}
```

Keys are station IDs (string in JSON, int in memory after load).
Values are grams at the BASE batch weight. The Dashboard scales them by `(selected_batch_g / sum(values))` before dispatch.

11.4 Inventory File (inventory.json)

```
{
  "1": { "capacity": 1000.0, "stock": 800.0 },
  "2": { "capacity": 1000.0, "stock": 960.0 },
  ...
}
```

stock is reduced by ACTUAL grams (not target) after each fill.
Below LOW_STOCK_THRESH (100 g) the row turns amber.
Refills bump stock and append a row to refill_log.csv.

11.5 Ingredient Catalogue (ingredients_catalogue.json)

Master list of ingredient names available in the Station Manager dropdown. Seeded from factory defaults; deduped and sorted on save. `add_ingredient_to_catalogue(name)` is the helper used by the UI when a new ingredient name is typed in the Behavior tab.

11.6 Production Log Columns

Column	Type	Meaning
OrderID	int	Auto-incrementing, persists across reboots (next_oid)
Timestamp	ISO 8601 s	Local time at row write
Recipe	str	Recipe display name at dispatch time
Station	int	Station ID (1..13)
Tea	str	Station label at dispatch time (INGREDIENTS[sid]['label'])
Target_g	float	Recipe target weight (g)
Actual_g	float	Smoothed weight after the lane completed (Station.actual)
Delta_g	signed float	Actual – Target
Tolerance_g	±float	From tolerance(target) = max(0.2, 1% of target)
Status	str	OK WARN EJECTED CANCELLED ERROR

11.7 Refill Log Columns

Column	Meaning
Timestamp	ISO 8601 s
Station	S01..S13
Ingredient	Station label
Added_g	Refill amount (operator entered)
NewStock_g	Post-refill stock (clamped to capacity)

12. Engineering Decisions & Tradeoffs

12.1 Why a multi-board topology?

Four Motion Pro boards instead of one Pi-side GPIO breakout: gives fault isolation (one stuck HX711 cannot block the others), keeps motor noise off the Pi PCB, and lets the system scale to 16 stations (or 20) by adding another board + udev rule.

12.2 Why micro dispensing exists

Continuous bulk dispensing at a fixed cruise voltage cannot land small targets (≤ 5 g) accurately: by the time the load cell registers the target weight, the auger has already delivered another tick's worth of material. Pulses (motor on for 20–110 ms, then off, then settle, then read) make every dosing increment independently verifiable. Trade: speed — a 4 g rose-petal fill takes ~ 20 pulses and $\sim 20 \times (\text{pulse_ms} + \text{settling_delay_s}) \approx 25$ s, vs ~ 1 s for an 85 g BOPF fill in normal mode.

12.3 Why `fall_delay_s` and `settling_delay_s` matter

Material falling from an auger has non-zero settling time on the load cell — the cell is a spring-mass system with damping; a freshly dropped batch oscillates around its true weight for hundreds of milliseconds. Reading the scale during that ringing under-counts the delivered amount and causes the loop to over-fill. The two settle constants — `fall_delay_seconds` (legacy, used by orchestrator WEIGHING) and `settling_delay_s` (used between bulk-stop and FINAL_MICRO) — bound this error.

12.4 Why station-specific behavior

BOPF, Peko, BOP have predictable flow rates; rose petals and jasmine clump and bridge in the auger; cinnamon chips tumble in irregular chunks; bergamot is a sticky flavour pellet. A single set of dispense constants cannot serve all 13. Per-station tuning in `station_config.json` lets the same firmware and same orchestrator dispense radically different ingredients correctly.

12.5 Why conveyor + mixer synchronisation

If the servo flap opens before the conveyor moves, the first batch stalls on a static cup and may not catch the mixer head. If the mixer starts before the first valid drop, it spins dry. The chosen sequence — conveyor on at recipe start; mixer on at first valid drop; both off after `FINAL_MIX_60_SECONDS` — minimises wear and guarantees every drop lands on a moving belt.

12.6 Why per-station tolerance

A ± 0.2 g tolerance for an 85 g BOPF fill is meaningless (under 0.25 %). A ± 3 g tolerance for a 4 g petal fill is too loose (75 %). Per-station `scale_tolerance_grams` lets each lane have its own band. The `CSV-level tolerance()` function uses `max(0.2, 1% of target)` as a second indicator for the OK/WARN status column on disk.

12.7 Why servo drop + tap + eject

Drop = pour the dispensed material onto the conveyor. Tap = oscillate the flap 3× between 45° and 70° to dislodge ingredient still sticking to the chute wall (especially relevant for chunky and sticky ingredients). Eject = backflip -70° to route an out-of-tolerance batch into an inside reject basket without contaminating the recipe.

12.8 Why invalid-weight rejection (operator modal)

Automatic decisions for out-of-tolerance batches risk dosing errors downstream (recipe is short of a key ingredient and the operator is unaware). A modal forces a human decision: EJECT to inside basket (safe, recipe is PARTIAL); RETRY (re-run only this lane); CANCEL (abort recipe). 10-minute timeout defaults to CANCEL so a distraction does not stall the line forever.

12.9 Why a single CSV log file

production_log.csv is the system of record. One file, one schema, trivially exportable to Excel / Google Sheets / a downstream MES. Each row is one ingredient (not one order), so per-station yields and per-recipe completeness fall out of a simple GROUP BY.

12.10 Why JSON (not SQLite) for configs

station_config.json / recipes.json / inventory.json / ingredients_catalogue.json are small (kB-scale), need to be hand-editable when the UI is unavailable, and benefit from atomic-write patterns with no schema migration risk. SQLite would add a hard dependency and gain little over flat JSON at this scale.

13. Safety & Reliability Engineering

13.1 Layered Watchdogs

Layer	Mechanism	Trigger	Action
Firmware	code.py main loop checks $(\text{now} - \text{last_cmd_time}) > 0.7 \text{ s}$	No valid serial command in 700 ms	stop_all_motors() — every motor PWM to 0
Pi heartbeat	Board.poll() sends W:ALL every WATCHDOG_INTERVAL_S = 500 ms	Always — proactive	Keeps firmware watchdog reset
Soft-stop	Board.poll() sees no valid response for HARD_WD_S = 700 ms	Board lost / OS hung the serial	Broadcast X to every board; log SOFT-STOP
Operator	■ E-STOP top-bar button	Manual	orch.abort() + estop_all() (broadcast X), queue purged

13.2 Motor Safety

- PWM at 20 kHz (above human hearing, above mechanical resonance) minimises acoustic noise and prevents the motor from doubling as a vibrator that corrupts the load cell.
- VOLT_MIN/VOLT_MAX (2.0 V / 6.0 V) clamp cruise voltage to the safe operating band of the auger gear motors. The duty calculation rounds 65535 to int after clamping to 12 V.
- Manual diagnostics (test_motor_pulse) are capped at MAX_MANUAL_MOTOR_S (3 s) and MANUAL_TEST_VOLTAGE (2.5 V), and refuse to run if an orchestrator recipe is in progress.
- motor_on(sid) / motor_off(sid) record cumulative runtime per station for predictive maintenance (motor_hours()). MAINT_WARN_HOURS (500 h) raises a yellow alert in the Health tab.

13.3 Servo Safety (recap)

- Always detach after motion (S:<ch>:-1) — no holding current.
- Per-station safe angle envelope clamps every write.
- Cooldown (30 s active / 60 s window → 5 s rest) prevents thermal stall.
- Direction-change pause (50 ms) absorbs reversal current spikes.
- Rate limit (5 ms between writes) prevents bus flooding.
- Per-sequence movement timeout (8 s default) aborts stuck sweeps.
- Always-detach in finally clauses around _drop_t and _eject_blocking.

13.4 Invalid-Dispensing Protection

- Per-lane WEIGHT_VALIDATION ($0 \leq \text{delta} \leq \text{scale_tolerance_grams}$) before any drop. Out-of-tolerance lanes never drop without operator consent.
- Stock pre-validation (check_stock) blocks a recipe before any motor energises if any hopper is short.
- _modal_lock serialises operator-decision modals across lanes; the all-fills-done barrier guarantees the modal cannot interrupt an active dispense.
- Stall detection: 3 consecutive top-up or rescue pulses with $< 5 \text{ mg}$ gain exit the loop and propagate as 'short' downstream; the operator sees an out-of-tolerance modal instead of an infinite spin.

13.5 Overload Protection (HX711)

- ERR_RETRY (3) auto-tare recovery: after three consecutive 'ERR' readings on a channel, code.py auto-tares that channel. The Pi logs 'auto-tare'. If the cell is physically overloaded, the next read will still be ERR until

physically corrected.

- Spike rejection (`MAX_JUMP_G` = 25 g; `SPIKE_TOLERATE_N` = 3) prevents single-sample EMI/RF spikes from triggering an early cutoff. A real step change (operator placed a calibration weight) is accepted after three consecutive jumps in a row.

13.6 Operator Safety

- E-STOP top-bar button is always visible, always reachable. Pressing it both aborts the orchestrator and broadcasts the X command to every board.
- Tech-mode PIN (2350) gates Behavior, Recipe Change, and Diagnostic tabs — floor operators cannot accidentally drift calibration or trigger raw motor pulses.
- Toast notifications + status bar keep the operator informed of queue state, partial outcomes, and ejected lanes without modal blocking.
- The dashboard refresh button asks for confirmation when active dispensing is in progress.

13.7 Recovery Methods

Symptom	Recovery
Board offline (red LED on dashboard)	Health tab → Refresh; check udev rules + USB cable; verify <code>ls /dev/module_0X</code>
Weights stuck at 0.00 g	Per-station T button (tare); manual <code>T:<ch>\n</code> from a serial terminal; check 3V3 + GND on HX711 wiring
Motor doesn't run	Diagnostic tab → MOTOR ON; verify 12 V PSU; verify GPIO mapping in <code>code.py</code>
Servo doesn't move	Diagnostic tab → DROP TEST; verify 5 V buck on dedicated rail; verify GPIO mapping
Conveyor/Mixer unresponsive	Confirm Board 4 online; manual <code>D:2:52000 + D:3:40000</code> from a serial terminal
Consistent overshoot	Decrease pulse durations and/or increase <code>fine_margin_g/inflight_compensation_g</code> ; allow <code>learned_compensation_g</code> to converge
Consistent undershoot	Increase <code>max_topup_pulses</code> (cap 50); rescue pulses already engage; verify auger not bridged
Dispatch hangs in DISPENSING	One lane has <code>_running</code> but no flow → check station status badges; press ■ STOP on the stuck station; then ■ E-STOP
<code>production_log.csv</code> not created	Verify write permission on the app directory; <code>ensure_log_file()</code> will recreate it on next <code>log_production()</code> call

14. Performance & Precision Analysis

14.1 Dispense Time (typical)

Scenario	Estimated time
85 g BOPF in normal+settle+final_micro	≈ 12–18 s (bulk ~10 s + settle 1 s + FINAL_MICRO 3 g)
10 g additive in auto (normal path)	≈ 5–8 s
4 g rose petals in PURE MICRO	≈ 20–25 s (≈ 20 pulses × pulse_ms + settling_delay_s)
Full Ceylon Spiced Breakfast (1+6+7)	≈ max(BOPF, cinnamon, ginger) due to parallelism; ~18 s + 1.5 s conveyor ramp + 60 s mix = ~80 s end-to-end

14.2 Precision (per-lane)

- Fill loop targets [target, target + UPPER_TOL_G] where UPPER_TOL_G = 0.05 g. The orchestrator's WEIGHT_VALIDATION accepts up to scale_tolerance_grams (default 3.0 g) and ONLY non-negative delta — i.e. the loop never deliberately leaves a lane short.
- Effective per-sample noise on the load cell: ≈ 5–20 mg (1-σ) after median + EMA. Practical landed accuracy goal: ±0.05 g.
- Adaptive learned_compensation_g closes the loop over multiple fills, reducing residual bias to roughly zero after 3–5 fills of the same ingredient at similar target weights.

14.3 Latency Budget

Hop	Latency
HX711 conversion → DOUT LOW	≈ 100 ms typical at 10 SPS hardware rate
Bit-bang 24 bits + gain pulses	≈ 50–100 μs
read_all_weights() over 4 channels	≈ 1–4 ms wall (most channels are already ready)
Serial WA: round-trip Pi↔Board	≈ 5–25 ms typical (latency_ms tracked per board)
Board.poll() tick	20 ms
Smoothing pipeline	Negligible (< 1 ms)
Motor cutoff → flow stops	SYSTEM_DELAY_S ≈ 80 ms (modelled in fill loop)
Material in-flight after cutoff	modelled via flow_rate × SYSTEM_DELAY_S + inflight_compensation_g + learned_compensation_g

14.4 Overshoot / Undershoot Behaviour

- Overshoot prevention: predictive cutoff in COARSE (slow_start ramp), predictive 'next pulse would exceed' check in MICRO/FINAL_MICRO, stable_read uses LATEST reading not pre-pulse, learned_compensation_g accumulates a per-station bias.
- Undershoot rescue: TOP-UP loop (up to 30 PULSE_VOLTAGE pulses), then RESCUE loop (up to 8 TOPUP_RESCUE_VOLTAGE pulses at 3.4 V). Triggers at TOPUP_RESCUE_TRIGGER_G (0.10 g).
- Stall guards exit the loop cleanly when the auger cannot deliver material (3 consecutive pulses with < 5 mg gain). The lane then exits READY_TO_DROP short of target; the orchestrator's validation raises the operator-decision modal.

14.5 Ingredient Variability

Per-pulse gain varies wildly across ingredients (BOPF ~200–400 mg/pulse vs petals ~30–80 mg/pulse). The auto-adapt pulse-duration table (110/60/35/20 ms by gap) and the extra-careful MICRO_FINE_GAP regime (tiny pulse × 0.8 V factor below 0.5 g) absorb most of this variability without per-ingredient code paths.

14.6 Scaling Limits

Limit	Bound	Why
Concurrent active lanes	13	13 hoppers; orchestrator parallelism scales linearly
Queue depth	5	Conservative for the dashboard scroller; can be raised in App._add_queue
Polling rate per board	~50 Hz	HX711 hardware sample at 10 SPS is the true ceiling
Operator decision window	600 s	Auto-CANCEL
Pulse budgets	$30 + 8 = 38$ / lane	After this many ineffective pulses the loop exits 'stalled'
Motor runtime alert	500 h	MAINT_WARN_HOURS — yellow Health alert

14.7 Known Accuracy Bottlenecks

- First-fill bias on a fresh container: `learned_compensation_g` starts at 0 (or whatever was persisted) and takes 3–5 fills to converge. Mitigation: persist `learned_compensation_g` across reboots (already done) and pre-seed reasonable values per ingredient family.
- Mechanical wear on the auger lowers per-pulse gain over time; MAINT_WARN_HOURS (500 h) is a coarse marker. A finer instrument would be flow-rate trending in `production_log.csv`.
- Hopper bridging (clumped petals refusing to flow) is detected as stall but not actively cleared by the firmware. Manual intervention is required.
- EMI from the mixer can momentarily bias load cell reads on Board 3 and Board 4 (geographically closest). `vibe_frozen()` suspends polling on those boards while the mixer is running; weight is read again after VIBE_FREEZE_S (2 s).

15. Future Improvements

15.1 Hardware

- Replace MG996R hobby servos with industrial Dynamixel-class digital servos that report load and temperature over a serial bus. Brings true closed-loop torque and overheat protection in firmware.
- Switch from 100 g hobby load cells to ratiometric industrial cells (e.g. 500 g or 1 kg, 0.02 %FS). Improves long-term repeatability and tolerates overload events without permanent drift.
- Replace bit-banged HX711 with a dedicated ADC daughter-board (e.g. ADS1232 or AD7193 modules) that integrate filtering and offer deterministic timing.
- Add hardware E-STOP latch (relay) wired in series with the 12 V PSU so the operator's E-STOP cuts main power, not just the duty cycle.
- Add per-station auger torque sensing (current sense on H-bridge) to detect a bridged hopper in real time rather than via stall counters.
- Add load-cell shielding (mu-metal can or Faraday enclosure) on Boards 3/4 to remove mixer EMI without needing `vibe_freeze`.

15.2 Software

- Per-station auto-calibration sequence: place known weight, capture raw HX711, write `REF_UNIT`, redeploy. Fully scripted instead of the current manual procedure.
- 'Train auger' mode: pulse N times at different voltages, log gain, fit a per-ingredient gain curve, persist it. Replaces hand-tuned `pulse_ms_*` values per station.
- Migrate the Tkinter HMI to a FastAPI + websocket + browser front end. Keeps the same backend module unchanged; gains remote operation and multi-operator visibility.
- Streaming export of `production_log.csv` to a Google Sheet / network share for daily roll-up.
- Multi-recipe queue scheduling with stock-aware re-ordering: pick the next recipe whose ingredients are all in stock first; defer those blocked on low stock.
- Confidence-weighted fill: track per-station residual variance and widen `scale_tolerance_grams` automatically for noisy lanes.

15.3 ML / Modelling

- Replace the static quadratic ML voltage model (-0.0677, 1.6538, 0.5615) with an online-trained per-station model fed by `production_log.csv`. Targets typically 1–95 g; the model output is a recommended cruise voltage given desired flow.
- Per-ingredient dwell model: for the `FINAL_MICRO` loop, learn the mean per-pulse gain and the variance, then choose pulse durations and counts that minimise expected absolute error within a fixed time budget.
- Predictive bridging detection: a sudden drop in per-pulse gain below the historical mean strongly suggests a bridged hopper; raise a maintenance alert before the stall counter fires.

15.4 Operational

- Barcode scanner integration for ingredient refill: scan a container, the UI auto-selects the station and pre-fills capacity / lot fields.
- OEE dashboard (availability × performance × quality) computed from `production_log.csv` and `refill_log.csv`.
- Daily 'tare all' scheduled task at shift start to clean thermal drift.
- Hot-swappable hopper containers with RFID — the controller automatically loads the right ingredient profile when a container is mounted.

16. Appendix

16.1 Glossary

Term	Meaning
BOPF	Broken Orange Pekoe Fannings — a high-grade Ceylon black tea grade
BOP	Broken Orange Pekoe — a Ceylon black tea grade
Peko	Pekoe — a black tea grade (whole leaf)
Silver / Golden Tips	Premium tip-grade Ceylon teas
Auger	Helical screw that meters ingredient out of a hopper
Hopper	Container above the auger holding the bulk ingredient
Bridging	Clumping of light/sticky material in the hopper that stops flow
ADC	Analogue-to-Digital Converter
EMA	Exponentially Weighted Moving Average
EMI	Electromagnetic Interference
GPIO	General Purpose Input/Output
PWM	Pulse-Width Modulation
RP2350	Raspberry Pi RP2350 dual-Cortex M33 microcontroller (Motion Pro core)
udev	Linux device-naming subsystem; rules in /etc/udev/rules.d
CDC	USB Communications Device Class (virtual serial port)
E-STOP	Emergency stop — single-action shutdown of all motion
HMI	Human-Machine Interface (the dashboard)
MES	Manufacturing Execution System
FS	Full Scale (load cell rating)
LSB	Least Significant Bit
ORD-NNNN	Order ID format used in production_log.csv (e.g. ORD-1001)

16.2 Acronym Quick Reference

Acronym	Expansion
HX711	Avia Semiconductor HX711 — 24-bit Σ - Δ load-cell ADC
MG996R	Tower Pro MG996R — metal-gear standard servo (~10 kg·cm)
MG90S	Tower Pro MG90S — micro metal-gear servo (~1.8 kg·cm)
PSU	Power Supply Unit
PD	USB-C Power Delivery
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
UDEV	Userspace Device Manager
TECH_PIN	Technician Personal Identification Number — gates locked tabs

16.3 Troubleshooting Quick Table

Symptom	First check	Then check
All boards offline	Is /dev/ttyACM* + udev rules	Pi user in dialout group; replug USB

Symptom	First check	Then check
One board offline	udevadm info -a -n /dev/module_0X	Specific cable / port; firmware deployed
Weights stuck at 0	HX711 3V3 + GND + CLK/DAT pins	Per-station T button; serial T:<ch>\n
Weights jitter ± grams	Vibration; servo back-EMF; loose load cell	vibe_freeze coverage; cable routing
Auger doesn't run	12 V PSU on; H-bridge enable	Diagnostic MOTOR ON; D:<ch>:32768 from serial
Servo doesn't move	5 V dedicated buck on; servo GPIO mapping	Diagnostic DROP TEST; per-station envelope
Conveyor/Mixer dead	Board 4 online; motor slots 2/3 wiring	CONVEYOR_DUTY / MIXER_DUTY constants
Chronic overshoot	fine_margin_g / inflight_compensation_g per station	learned_compensation_g; pulse durations
Chronic undershoot	max_topup_pulses; rescue trigger	Hopper not bridged; auger not worn
Operator modal not appearing	All-fills-done barrier — wait for slowest lane	Lane stuck FILLING — press STOP, then E-STOP
production_log.csv stale	ensure_log_file() runs at import	Write permission; SD card health
Order ID jumped or reset	_load_last_oid() reads bottom of CSV	CSV not corrupted/truncated

16.4 Calibration Guide (summary)

- Per-channel HX711 REF_UNIT: place 100 g weight → T:<ch> → raw = hx.read() → REF_UNIT = raw / 100.0 → write into code.py REF_UNITS list → redeploy.
- Per-station learned_compensation_g: leave at 0 and run 3–5 fills at a representative target. The fill loop will converge it automatically and persist via _persist_learned_comp().
- Per-station fall_delay_seconds / settling_delay_s: trial-and-error with the Diagnostic tab DISP TEST. Increase if FINAL_MICRO over-doses on the first pulse; decrease if dispense is unnecessarily slow.
- Per-station servo envelope: set safe_angle_min / safe_angle_max to the actual mechanical limits of the linkage; the firmware will refuse to drive outside that range and the Behavior tab refuses to save values outside the absolute limits SAFE_ANGLE_MIN_LIMIT (−90°) and SAFE_ANGLE_MAX_LIMIT (+120°).

16.5 Startup Flow

1. Pi boots → systemd unit teamatrix.service (optional) launches python3 teamatrix_pro.py with DISPLAY=:0
2. App.__init__ wires UI, registers app_ref into the backend, spawns init_boards() in a daemon thread
3. init_boards() calls _load_last_oid(), ensure_log_file(), then for each bid in BOARD_PORTS spawns Board.connect() and Board.poll() daemon threads.
4. Each Board.connect() opens pyserial, sleeps 1.5 s for USB reset, sends T:ALL, homes + detaches all servos, sets connected=True.
5. Board.poll() starts ticking at POLL_INTERVAL_S; weights populate.
6. App._loop() begins at GUI_MS = 40 ms.
7. Operator interacts.

16.6 Shutdown Flow

1. Operator presses Window-Close (X) or sends SIGTERM.
2. App._close handler is invoked.
3. orch.abort() → _abort.set(); release all-fills barrier; decision_event.set(); broadcast D:2:0, D:3:0; vibe_freeze off; every Station.stop() ensures motor duty 0.
4. estop_all() broadcasts X to every board.
5. Tkinter destroys widgets; pyserial threads detect _run=False and exit.

16.7 Maintenance Guide

- Daily: tare all scales at shift start (button on Dashboard).

- Weekly: visually inspect each hopper for bridging; clear with soft brush. Check servo flap motion through Diagnostic DROP TEST.
- Monthly: compare per-station learned_compensation_g values — values drifting $> \pm 1$ g often indicate mechanical wear (auger flute polish or hopper geometry change).
- 500 h motor runtime alert (MAINT_WARN_HOURS): inspect auger gearbox, check H-bridge thermal paste, audit servo backlash.
- Quarterly: recalibrate every HX711 REF_UNIT against a traceable 100 g calibration weight.
- After any wiring change: confirm udev rules still bind /dev/module_0X to the intended physical board.

16.8 Testing Checklist (pre-handover)

- All four boards show ONLINE green on the Dashboard.
- Latency per board < 50 ms (Health tab).
- Empty-platform stable weight is 0.00 ± 0.05 g on every lane after TARE ALL.
- 100 g calibration weight reads 100.00 ± 0.20 g on every lane.
- DROP TEST per lane: servo sweeps $0^\circ \rightarrow 70^\circ$, taps 3 $\times$, returns 0° , detaches; servo not warm to touch after the sequence.
- EJECT TEST per lane: servo sweeps $0^\circ \rightarrow -70^\circ$, holds 0.8 s, returns 0° , detaches.
- DISP TEST 5 g per lane: lane fills, settles, final weight within [5.00, 5.05] g (UPPER_TOL_G band) for normal mode; ≤ 0.10 g error for micro mode.
- Full recipe (Ceylon Spiced Breakfast 100 g): SUCCESS outcome, all rows in production_log.csv with Status=OK.
- Force out-of-tolerance: temporarily set scale_tolerance_grams = 0.05 g; recipe should raise the operator modal; verify EJECT/RETRY/CANCEL all behave correctly; restore tolerance.
- E-STOP test: during DISPENSING, press E-STOP; verify all motors stop within 1 s; queue is purged.
- Power-cut test: hard-cut power during DISPENSING; on reboot, production_log.csv has the partial row(s) (CANCELLED/ERROR), and the next order ID continues correctly.

16.9 File-Path Reference

File / Path	Purpose
/dev/module_01..04	Stable udev symlinks per board (via 99-tea-lanes.rules)
/etc/udev/rules.d/99-tea-lanes.rules	udev binding (idVendor 239a, idProduct 8111)
/etc/systemd/system/teamatrix.service	Optional auto-start unit
<app>/station_config.json	Per-station Behavior values
<app>/recipes.json	Saved recipe library
<app>/inventory.json	Container stock
<app>/ingredients_catalogue.json	Available ingredient names
<app>/production_log.csv	Per-ingredient order log
<app>/refill_log.csv	Manual refill events

16.10 Reference Implementations (verbatim file lines)

Authoritative parameter values are at the top of teamatrix_backend.py. Selected verbatim:

```
BOARD_PORTS = {1: '/dev/module_01', 2: '/dev/module_02',
                3: '/dev/module_03', 4: '/dev/module_04'}
BAUD_RATE = 115200; POLL_INTERVAL_S = 0.02; WATCHDOG_INTERVAL_S = 0.5
MIXER_DURATION_S = 60; CONVEYOR_CH = 2; MIXER_CH = 3
CONVEYOR_DUTY = 52000; MIXER_DUTY = 40000
```

```

SUPPLY_VOLTAGE = 12.0
ML_A, ML_B, ML_C = -0.0677, 1.6538, 0.5615
VOLT_MIN, VOLT_MAX = 2.0, 6.0; ACCEL_STEP = 0.2
PULSE_VOLTAGE = 2.8; PULSE_DURATION = 0.04
PULSE_WAIT = 0.6; PRE_ACT_TIME = 0.12
EMA_ALPHA = 0.7; DEAD_ZONE = 0.01; ZERO_RANGE = 0.50
MED_BUF_LEN = 5; MAX_JUMP_G = 25.0; SPIKE_TOLERATE_N = 3
HARD_WD_S = 0.7; ERR_RETRY = 3; VIBE_FREEZE_S = 2.0
TECH_PIN = '2350'
MAINT_WARN_HOURS = 500.0; LOW_STOCK_THRESH = 100.0
DROP_TOLERANCE_G = 3.0
DEFAULT_DISPENSE_MODE = 'auto'
DEFAULT_MICRO_THRESHOLD_G = 5.0
DEFAULT_SCALE_TOLERANCE_G = 3.0
DEFAULT_FALL_DELAY_S = 2.0
DEFAULT_TARGET_OFFSET_G = 0.0; TARGET_OFFSET_MAX_G = 5.0
OPERATOR_DECISION_TIMEOUT_S = 600.0
DEFAULT_AUTO_MICRO_TAIL_G = 3.0; AUTO_MICRO_TAIL_MAX_G = 25.0
MICRO_TAIL_SLOWDOWN_G = 1.0
DEFAULT_FINAL_MICRO_AMOUNT_G = 3.0; FINAL_MICRO_AMOUNT_MAX_G = 10.0
DEFAULT_SETTLING_DELAY_S = 1.0; SETTLING_DELAY_MAX_S = 30.0
MICRO_FINE_GAP_G = 0.5; MICRO_FINE_VOLT_FACTOR = 0.8
DEFAULT_COARSE_MARGIN_G = 5.0
DEFAULT_FINE_MARGIN_G = 0.30; DEFAULT_INFLIGHT_COMP_G = 0.15
DEFAULT_LEARNED_COMP_G = 0.0; LEARN_ALPHA = 0.30
PRECISION_LEARNED_CLAMP_G = 3.0
DEFAULT_SETTLE_MS = 500; DEFAULT_STABLE_WINDOW_G = 0.05
DEFAULT_STABLE_SAMPLE_CNT = 4
DEFAULT_PULSE_MS_LARGE = 110; DEFAULT_PULSE_MS_MEDIUM = 60
DEFAULT_PULSE_MS_SMALL = 35; DEFAULT_PULSE_MS_TINY = 20
DEFAULT_MAX_TOPUP_PULSES = 30
TOPUP_RESCUE_PULSES = 8; TOPUP_RESCUE_VOLTAGE = 3.4
TOPUP_RESCUE_TRIGGER_G = 0.10
SERVO_DETACH_SETTLE_S = 0.15; SERVO_DIRECTION_CHANGE_PAUSE_S = 0.05
SERVO_MIN_CMD_INTERVAL_S = 0.005
SERVO_MOVE_TIMEOUT_S_DEFAULT = 8.0
SERVO_COOLDOWN_RUN_S = 30.0
SERVO_COOLDOWN_WINDOW_S = 60.0
SERVO_COOLDOWN_DURATION_S = 5.0
DEFAULT_SAFE_ANGLE_MIN = -90.0
DEFAULT_SAFE_ANGLE_MAX = 120.0
SAFE_ANGLE_MIN_LIMIT = -90.0
SAFE_ANGLE_MAX_LIMIT = 120.0
MAX_MANUAL_MOTOR_S = 3.0
MANUAL_TEST_VOLTAGE = 2.5
MANUAL_DISP_TEST_TARGET_G = 5.0

```

16.11 End of Document

This documentation reflects the repository state at generation time. All numeric defaults, ranges, GPIO assignments, dispense phases, and protocol commands were extracted directly from `teamatrix_pro.py`, `teamatrix_backend.py`, `code.py`, `hx711_gpio.py`, `station_config.json`, and `inventory.json`. For changes after the generation date, re-run `build_docs.py`.